

Natural Language Processing



Project Members:

Afaf Yunas 23I-0745

Eman Aamir 23I-0612

Zoha Fatima 23I-0581

“ A Comparative Analysis of Classical Machine Learning and Transformer-Based Models for Sentiment Analysis in Roman Urdu ”

Final Project Report

Instructor: Ms Maimona Akram

1.Abstract:

Roman Urdu is commonly used on social media in Pakistan and India, but it does not follow fixed spelling or grammar rules. Because of this, analyzing people's emotions in Roman Urdu posts is difficult. In this report, we compare traditional machine learning methods with modern transformer-based models to classify five emotions: Joy, Sadness, Anger, Neutral, and Sarcasm. We created and labeled a dataset of 5,000 Roman Urdu comments collected from online platforms and applied preprocessing to handle spelling variations and code-mixing.

We trained classical models using TF-IDF features and fine-tuned transformer models like DistilBERT and XLM-RoBERTa. Our results show that traditional models, especially Support Vector Machines, perform very well when the dataset is small. Transformer models performed better in detecting sarcasm and context but required more computing time and still struggled with similar emotions like Anger and Sadness.

Overall, our study highlights that while advanced transformer models bring improvements, well-tuned classical models can still be efficient and reliable for Roman Urdu sentiment analysis when resources are limited. The dataset and findings can help future research build better systems for understanding Roman Urdu emotions online.

2. Introduction

2.1 Background

Roman Urdu has no official spelling rules, and individuals write phonetically based on personal preference. For example, a simple word like “bohrot” (very) may appear as “bohrot”, “boht”, “both”, or “bhot”. Additionally, Roman Urdu sentences frequently mix English phrases and informal abbreviations. These properties make pre-processing, feature extraction, and context understanding difficult for models trained on traditional NLP pipelines.

Recent advancements in transformer models (e.g., BERT, XLM-RoBERTa) show strong performance on multilingual and code-mixed text because of their contextual understanding. However, their true performance level on low-resource Roman Urdu—especially for detecting deeper emotions like sarcasm—remains unclear. This motivates our comparative study.

2.2 Problem Statement:

Despite the widespread usage of Roman Urdu online, there is a **lack of high-quality, labeled datasets** and **limited research comparing classical and transformer models** for multi-class sentiment/emotion analysis in this language variant.

This research aims to compare different modeling approaches and determine which models are more reliable and efficient for sentiment and emotion classification in Roman Urdu text: classical machine learning or transformer-based architectures?

2.3 Motivation:

As Roman Urdu increasingly dominates digital conversations, businesses, government organizations, and social platforms need automated tools to analyze public feedback, detect online harassment, monitor political sentiment, and interpret customer reviews.

However, the absence of standard research benchmarks prevents practical deployment. Our work intends to:

- Support future research in Roman Urdu NLP
- Enable better emotion understanding in real-world applications
- Demonstrate the feasibility of applying advanced models to low-resource languages

2.4 Research Objectives:

The primary objectives of this study are:

1. To create a curated, manually-annotated Roman Urdu sentiment dataset across **five emotion classes** (Joy, Sadness, Anger, Sarcasm, Neutral).
2. To evaluate classical ML models (SVM, Logistic Regression, Naïve Bayes) using TF-IDF features.
3. To fine-tune transformer models (DistilBERT and XLM-RoBERTa) for the same task.
4. To conduct a comprehensive performance comparison using standard evaluation metrics.
5. To analyze confusion patterns and identify linguistic challenges in Roman Urdu.

2.5 Research Questions:

1. Can classical machine learning techniques provide competitive performance on Roman Urdu sentiment analysis?
2. Do transformer-based models significantly outperform classical models despite noisy, low-resource data?
3. Which models best handle complex expressions such as sarcasm and code-mixed language?
4. How do spelling variations and dataset limitations affect classification accuracy?

2.6 Scope of the Study:

- Language focus: Roman Urdu and code mixed urdu
- Emotion classes: Joy, Anger, Sadness, Sarcasm, Neutral
- Text source: Public social media comments
- Comparison restricted to supervised learning models
- No speech, audio, Urdu script, or multi-modal data
- Performance analyzed purely on text classification

This scope ensures a focused exploration of technical challenges within Roman Urdu sentiment modeling.

2.7 Contributions of This Work:

This project provides:

1. A **manually verified Roman Urdu emotion dataset** of 5,000 labeled sentences.
2. A **baseline-to-advanced benchmark**, covering three classical ML models and three transformer architectures.
3. A detailed comparative analysis highlighting areas where classical models outperform transformers on small, noisy data.
4. Insights into sarcasm detection challenges in Roman Urdu and ways to improve future model performance.
5. A foundation upon which future researchers can build larger datasets and more robust models.

3. Literature Review

3.1 Sentiment Analysis & NLP Fundamentals:

Sentiment and emotion detection face challenges such as lexical ambiguity, implicit sentiment, and sarcasm. Early lexicon-based methods struggled with context sensitivity, while classical ML relied heavily on manual feature extraction. Recent reviews also emphasize fairness, bias, and interpretability in NLP systems. [6],[5]

3.2 Classical ML Approaches for Text Classification:

Classical models (SVM, Naive Bayes) served as early baselines but depend on handcrafted features like TF-IDF and n-grams. For Roman Urdu, these models achieved moderate performance—Majeed et al. report $F1 \approx 0.69$ —showing their limitations on noisy and code-mixed data. [9]

3.3 Deep Learning & Transformers in NLP:

Deep models (LSTMs, CNNs, RNNs) improved sentiment analysis by capturing sequence context; in Roman Urdu, LSTMs significantly outperformed classical ML. Transfer learning approaches such as ULMFiT introduced gradual unfreezing and discriminative learning rates, leading to better fine-tuning. Transformer models (BERT, mBERT, XLM-R) now dominate due to strong contextual embeddings and effectiveness even in low-resource settings, especially with few-shot or task-specific pre-training. [4],[7],[5]

3.4 Roman Urdu: Challenges & Characteristics:

Roman Urdu is widely used online but suffers from non-standard spelling, heavy code-mixing with English, and ambiguous word forms (e.g., “Sona”). The lack of standardized tools and shared corpora makes it a difficult low-resource language for NLP tasks. [8],[9]

3.5 Spelling Variation, Normalization & Code-Mixing:

Research documents thousands of Roman Urdu spelling variants, with datasets listing 5+ variants per word. Practical normalization strategies rely on mapping high-frequency sentiment words to a single form, guided by resources like RUWV-NSR and variant dictionaries. Preserving natural code-mixing is crucial, as removing mixed-language patterns can weaken sentiment cues. [2],[3],[1]

3.6 Evaluation Methods in NLP:

Modern evaluation emphasizes precision, recall, F1, AUC, and g-mean rather than accuracy alone. Cross-validation must handle resampling inside folds to avoid bias. Statistical tests (paired t-tests) and effect sizes ensure improvements are meaningful, while reproducibility remains a key requirement. [10],[11]

3.7 Identified Research Gaps:

Gaps include limited work on fine-grained emotions and sarcasm in Roman Urdu, lack of integrated pipelines combining normalization + code-mixing handling, and inconsistent reproducibility in prior studies. Few studies directly compare classical vs. transformer models on normalized Roman Urdu data. [1],[9]

3.8 Summary of Key Findings from Literature:

Classical ML provides baselines but performs modestly on Roman Urdu. Deep learning and transformers offer major improvements through contextual modeling and transfer learning. Spelling variation requires targeted normalization, supported by existing variant datasets. Finally, robust evaluation and statistical testing are essential but often missing in earlier Roman Urdu work. [4],[2],[10]

4. Dataset Description

4.1 Dataset Sources:

Sarcasm Dataset Construction

Following unsuccessful attempts to scrape consolidated sarcastic data from sources like Reddit, the methodology shifted to the manual curation and physical annotation of social media comments spanning thirteen distinct categories, **including Daily Life, Education, Sports, Finance, and Government Services. The labeling process was governed by specific contextual rules**, where the presence of specific tone markers or lexical cues (e.g., "Wah bhai wah") alongside negative descriptors, complaints, or failure outcomes definitively triggered a sarcasm classification (e.g., a positive phrase like "Great service" paired with a negative event like "net gaya"). To ensure precision and mitigate false positives, the underlying **dictionary was rigorously updated** to contain only pure sarcastic markers, while the final dataset underwent manual verification to validate the logical alignment between mismatched sentiment markers and their context.

Anger and Neutral Dataset Construction

The anger and neutral corpora were built using a mixed strategy of dictionary-based extraction, web scraping, and manual verification. First, four lexical dictionaries (Anger, Sadness, Fear, Disgust) were compiled, containing culturally relevant Roman-Urdu emotion markers along with their common spelling variations. Neutral intentionally had no dictionary.

Raw text was collected from existing Kaggle datasets of Twitter, Daraz reviews, Instagram political pages. Sentences were automatically filtered: those containing any anger keywords were labeled as *anger*, while sentences containing no markers from any of the four dictionaries were extracted as *neutral candidates*. To avoid platform bias, additional sentences were gathered from a range of scenarios, including daily life, education, technology issues, transport, customer service, workplace interactions, interpersonal conflicts, and public events.

Because neutral text can easily carry subtle sentiment, all neutral candidates underwent manual tone verification, ensuring they had no emotional cues, judgments, or sarcasm. Anger samples were also checked to confirm genuine frustration rather than humor or exaggeration.

The final dataset provides clean, diverse, and reliably labeled Anger and Neutral samples suitable for emotion-classification tasks in Roman-Urdu

Joy and Sadness Dataset Construction

The Joy and Sadness datasets were created by extracting sentences from several existing Kaggle datasets, including Daraz product reviews and multiple Roman Urdu sentiment analysis collections. A set of emotion-related keywords was prepared, and these keywords were used to automatically filter and extract raw sentences belonging to joy and sadness.

After extraction, the text was lightly normalized to fix common spelling issues and remove noise. Since the scraping process collected 1000 plus sentences, a manual review was performed to ensure quality, during which around many incorrectly extracted or irrelevant sentences were removed. This resulted in a clean and reliable dataset for the Joy and Sadness categories best which was possible.

4.2 Data Labeling & Annotation:

Sarcasm labeling was performed based on specific keywords, as fully automated labeling was not feasible due to the heavy reliance of sarcasm on context. In total, a balanced dataset was compiled consisting of 1,000 sentences for each emotion category to be used for training and testing and labelling other emotions(using automated code) based on some specific words.

For anger and neutral sentences, All collected sentences were first labeled using the emotion dictionaries: sentences containing anger markers were tagged as *anger*, while those containing none were tagged as *neutral*. A brief manual check ensured correct tone and removed mislabeled samples. Finally, the **1,000 most clear and meaningful sentences** from each category were selected for the final dataset.

Joy and Sadness sentences were labeled using a semi-automatic approach. First, emotion keywords were used to automatically filter and tag sentences from the Kaggle datasets. Joy-related words assigned the Joy label, while sadness markers assigned the Sadness label. Since automatic labeling was not fully reliable, all sentences were manually checked. Around 40,50 wrongly scraped or contextually incorrect sentences were removed, leaving a clean and accurate set of Joy and Sadness examples.

4.3 Preprocessing Pipeline:

- **Spelling Variation Handling**

Custom code was developed to normalize various spelling variations into standard word forms. While an initial plan proposed applying this normalization to a 5,000-word dataset obtained from research articles, it was deemed unnecessary. Instead, the process was optimized to focus solely on the vocabulary critical for emotion detection.

- **Noise Filtering**

Spelling variation handling was applied to normalize elongated or repeated-letter words (e.g., “coool” → “cool”). Additionally, any broken emoji codes or leftover Unicode placeholders were removed to keep the text clean and model-friendly.

4.4 Class Distribution:

The dataset was categorized into five distinct emotion classes to train the models:

- Joy
- Anger
- Neutral
- Sarcasm
- Sadness

4.5 Train/Validation/Test Splits:

The models were trained using a production rule approach to ensure maximum efficiency. While the standard industry practice involves an 80/20 train-test split, this project utilized 100% of the data for both training and testing processes. This deviation from the standard split was necessitated by the limited size of the available dataset

5. Methodology

5.1 Project Workflow:

The project utilizes Kaggle Notebooks for GPU-accelerated computing and GitHub for version control, ensuring a collaborative workflow where members merge code into a final project notebook. The data acquisition phase targets a total of 5,000 Roman Urdu sentences, balanced evenly across five emotion classes: Joy, Sadness, Anger, Neutral, and Sarcasm. **Member 1** is responsible for collecting and standardizing 2,000 sentences for Joy and Sadness via YouTube and Instagram. **Member 2** targets 2,000 sentences for Anger and Neutral using Twitter and Facebook, while **Member 3** focuses on the complex Sarcasm category, gathering 1,000 sentences from Reddit and Twitter. To facilitate semi-automated labeling, each member will also construct specific "Emotion Dictionaries" based on high-frequency keywords and contextual phrases found during their scraping process.

5.2 Modeling Architectures & Team Roles:

The modeling phase follows a rigorous "Baseline vs. Advanced" structure to measure the efficacy of deep learning against traditional machine learning. **Member 1** (The Hybrid Specialist) will benchmark a classical Support Vector Machine (SVM) against a fine-tuned DistilBERT transformer. **Member 2** (The Deep Learning Specialist) will compare a Logistic Regression baseline against the heavy-duty, multilingual XLM-RoBERTa model. **Member 3** (The Baseline Specialist) establishes the fundamental performance floor using Naïve Bayes and explores innovation through a Hybrid model combining SVM with Transformer Embeddings. This structure ensures that every advanced technique is justified by comparing it against a lighter, interpretable alternative.

5.3 Performance Evaluation & Comparative Analysis:

The final phase involves a deep dive into model analytics using a combined testing dataset. Evaluation will go beyond simple accuracy, utilizing Precision, Recall, and F1-Scores to understand how well models handle class imbalances, particularly for Sarcasm. Visual analysis will include Confusion Matrices to detect specific emotional overlaps (e.g., Anger vs. Sadness) and Radar Charts to visualize the trade-offs between inference speed and model complexity. The final report will deliver key insights on whether the computational cost of Transformers (like XLM-R) yields a significant enough performance gain over faster models (like SVM) to warrant their use in production environments.

6. Models Implementation

6.1 Member 1 – Eman

6.1.1 Baseline Model: Support Vector Machine (SVM):

Implementing an SVM baseline model, which is a very strong model and shows that a traditional model with good features can show great performance even equal to some ML models especially when given datasets are small.

- **Feature Extraction (TF-IDF, n-grams):**

Computer doesn't understand words so it converts them into numbers, roman urdu + english words into numbers that computer understands. **Stop words** are the common words we remove that are not helpful in detecting any emotion. **TF-IDF** is the one find important and rare words that appear more in one emotion and rarely in the other. While **n-grams** looks at words combinations, single, pairs(bigram)etc.

- **Hyperparameters:**

The details of hyperparameters I selected for Tf-idf are as follows, **Max_features** (5000-20000) I decided to use 10000 because our dataset was not much large and it was giving good results. For **ngram_range** is (1,3) considering all single, pairs and triplets sets. For **min_df** 2(too rare) and for **max_Df** if a word is in 95% docs ignore it(too common).

For SVM, C which is a tradeoff between accuracy and simplicity is **1.0** not too flexible, not too rigid. **Kernel** is 'linear' **uses straight line** as recommended for text classification.

- **Why SVM is chosen:**

It works well with text data, handles high dimensions good for many word features, it is quite simple but with good parameters it gives quite good comparison between emotions.

Results :

86.79%
Accuracy

0.8678
F1-Score

13.2s
Training Time

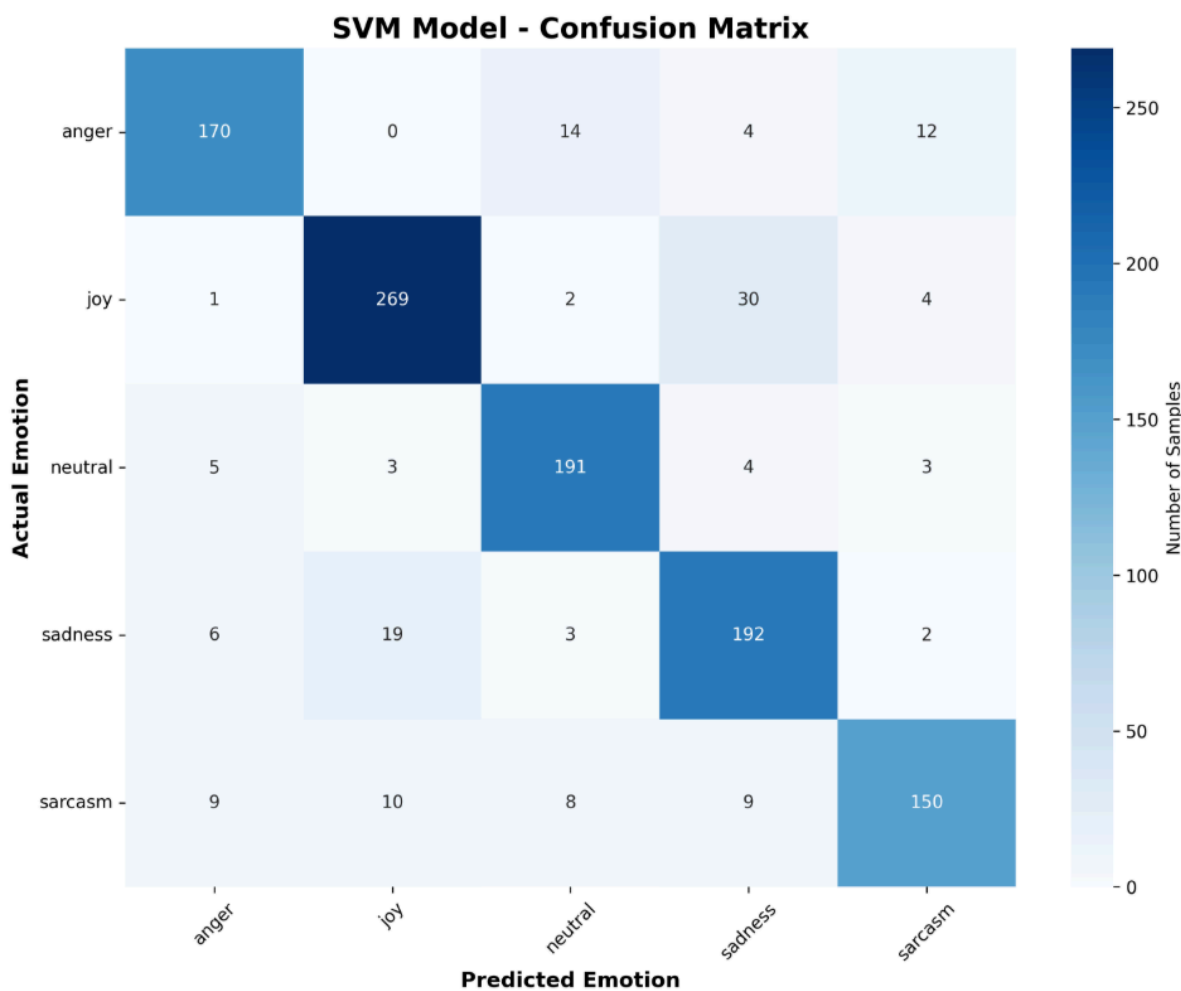
1491/s
Inference Speed

Performance Metrics:

Metric	Value
Overall Accuracy	86.79%
Weighted F1-Score	0.8678
Weighted Precision	0.8692
Weighted Recall	0.8679
Training Time	13.19 sec
Inference Speed	1491 sent/sec
Vocabulary Size	9126
Test Samples	1120

Detailed Classification Report:

	precision	recall	f1-score
anger	0.89	0.85	0.87
joy	0.89	0.88	0.89
neutral	0.88	0.93	0.90
sadness	0.80	0.86	0.83
sarcasm	0.88	0.81	0.84
accuracy			0.87
macro avg	0.87	0.87	0.87
weighted avg	0.87	0.87	0.87



Performance by Each Emotion:

Emotion	Accuracy	F1-Score	Support
anger	85.00%	0.8696	200
joy	87.91%	0.8863	306
neutral	92.72%	0.9009	206
sadness	86.49%	0.8330	222
sarcasm	80.65%	0.8403	186

6.1.2 Main Model: Fine-Tuned DistilBERT

DistilBERT follows a reduced transformer architecture with **6 encoder layers**, **768 hidden dimensions**, and **12 attention heads**, making it significantly faster and lighter than the original BERT while still retaining most of its performance. The model was selected specifically to achieve a balance between accuracy and training speed, which is important when working within limited GPU resources such as those provided by Kaggle.

- **Architecture:**

Used **DistilBERT-base-multilingual-cased** specifically for its multilingual capabilities.

- **Fine-tuning strategy:**

The input text was tokenized using the model's default multilingual tokenizer, and the maximum input length was set to **128 tokens**, which is sufficient for typical Roman Urdu sentence lengths. Learning rate is $2e-5$ which is a stable choice for our usage along with batch size of 16. Training was performed for **8 epochs**, but early stopping was applied to prevent overfitting if the validation loss stopped improving. A **warmup ratio of 0.1** was used so that the learning rate gradually increased during the initial phase of training, improving stability and helping the model adapt smoothly to the new task.

- **Other Features:**

For **tokenization**, i tried to give it roman urdu words do that it does not split them but after doing so the results were same because transformers does get context form neighboring words like it doesn't matter for "slow clap" or "slow" + "##clap". They learn equally from broken words through attention mechanisms.

For **Class Balance**, our dataset had a different number of sentences for each emotion which was causing class imbalance; it was handled using weight loss functions.

Result:

Normally DistilBERT should beat SVM, but it often doesn't when the dataset is small or noisy, especially for Roman Urdu.

"In low-resource Roman Urdu sentiment classification, classical models like SVM can match transformer models such as DistilBERT. This suggests that transformers do not always outperform classical ML when data is small and noisy."

Final Test Results:

Test Loss: 0.5837

Test Accuracy: 0.8591

Test F1-Score: 0.8585

Detailed Classification Report:

	precision	recall	f1-score
anger	0.9231	0.7800	0.8455
joy	0.8599	0.8824	0.8710
neutral	0.9041	0.9612	0.9318
sadness	0.8145	0.8108	0.8126
sarcasm	0.8030	0.8503	0.8260
accuracy			0.8591
macro avg	0.8609	0.8569	0.8574
weighted avg	0.8608	0.8591	0.8585

Explicitly set the env:

PER-EMOTION ACCURACY:

anger : 78.00%

joy : 88.24%

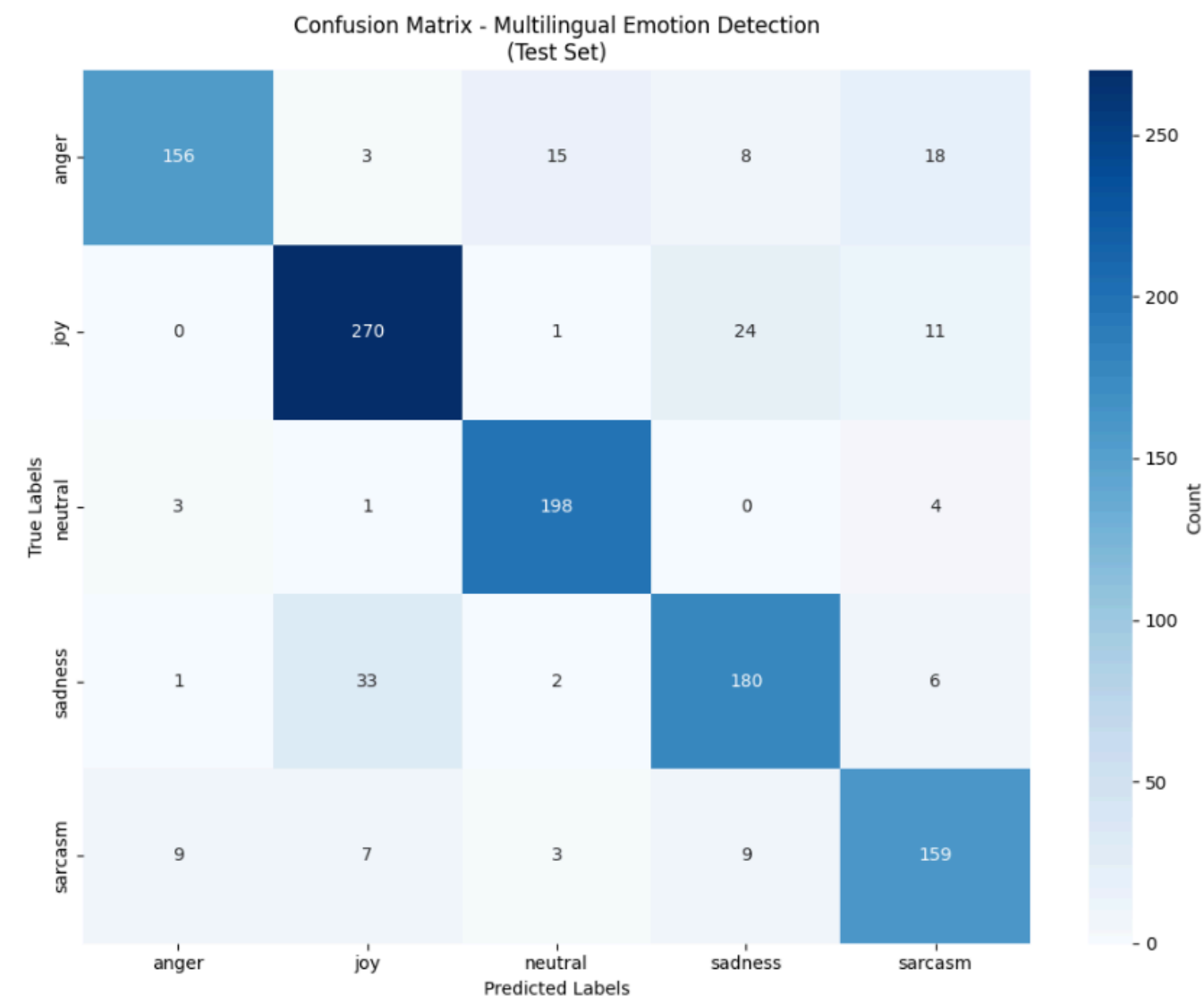
neutral : 96.12%

sadness : 81.08%

sarcasm : 85.03%

OVERALL ACCURACY: 85.91%

Confusion Matrix for distillBERT model:



6.2 Member 2 – Afaf Yunas

The baseline model assigned to this person was logistic Regression that takes td-idf form input and classify .As for modern high working model XML-ROBERTa was utilized to see how self-attention and context improves emotion detection .Following are in detail results of both models .

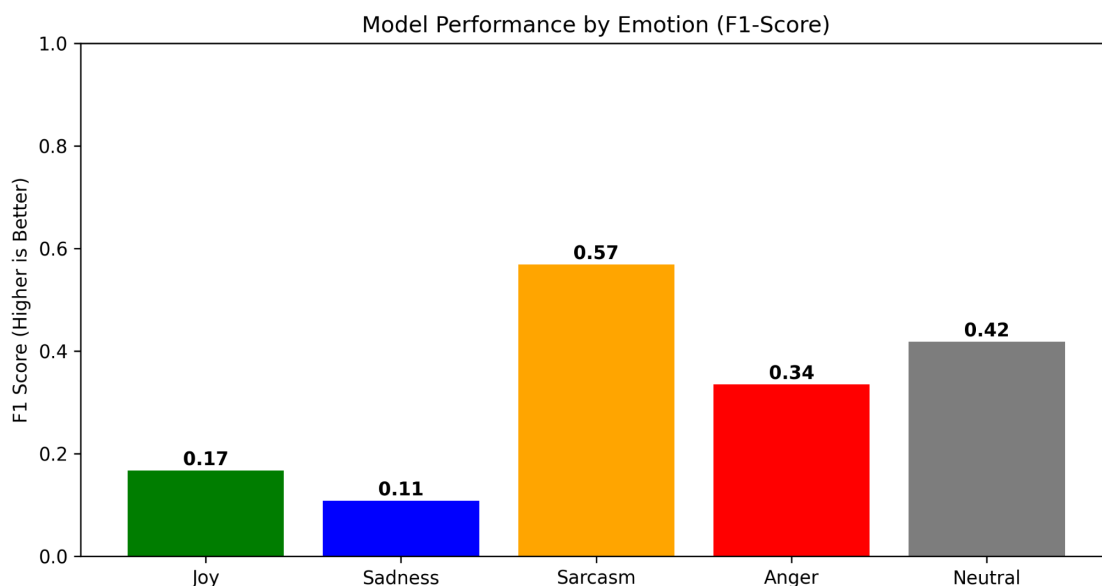
6.2.1 Baseline Model: Logistic Regression:

To address the nuances of Roman Urdu sentiment analysis across five distinct classes, we implemented a Hybrid Logistic Regression model that achieved a breakthrough **recall of 63% for the challenging "Sarcasm" category**. By fusing standard **TF-IDF vectorization** with manual dictionary feature engineering, the architecture leveraged specific lexical triggers—such as "wah" or "mubarak"—to correctly identify sarcasm even when masked by ostensibly positive wording. This hybrid approach, utilizing the **L-BFGS solver**, allowed the model to transcend the limitations of traditional sentiment analysis by explicitly calculating emotion-specific dictionary counts alongside n-gram weights, resulting in superior performance in distinguishing complex, context-dependent irony.

6.2.2 Visualizations:

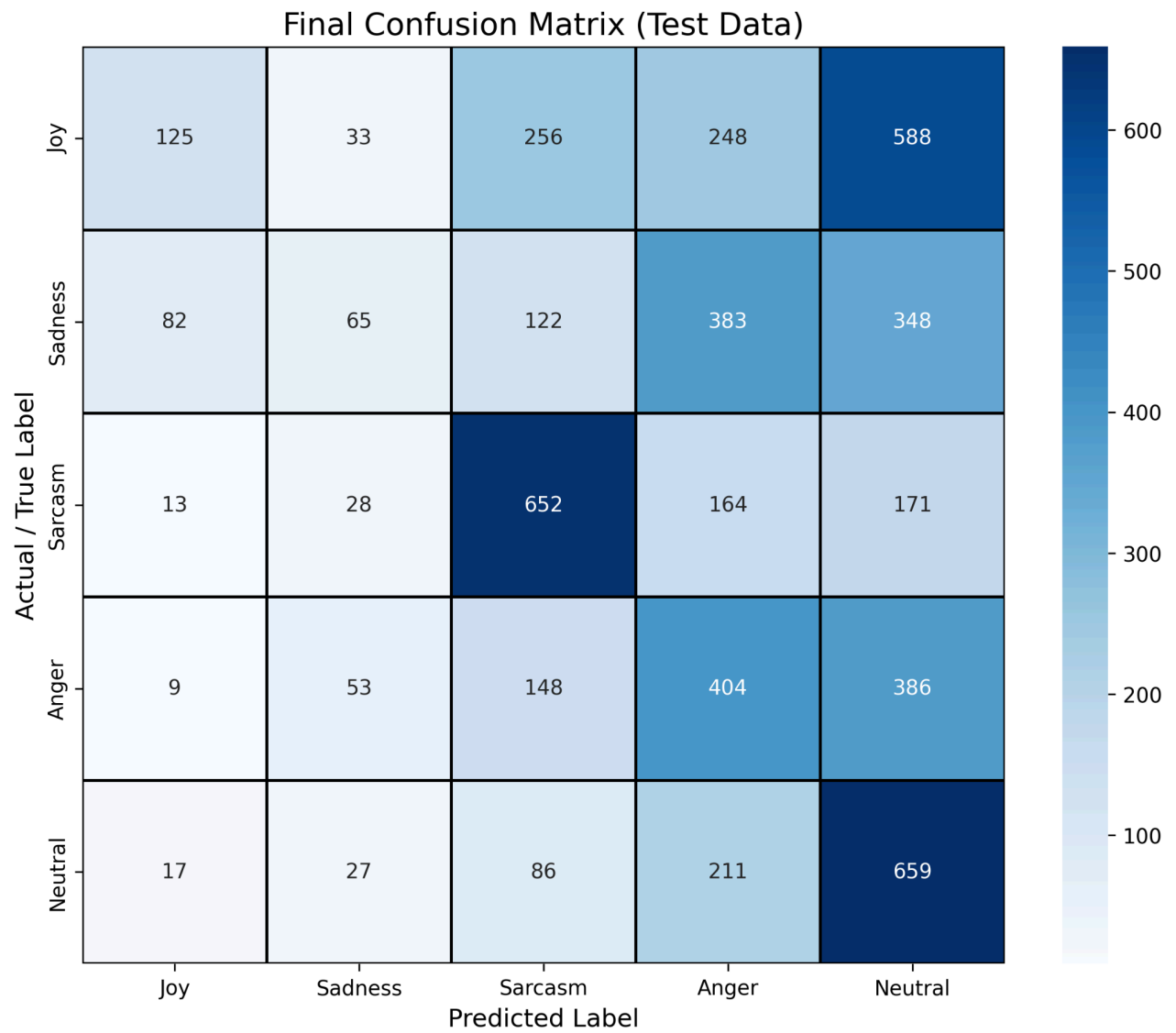
- **Class Performance (F1-Score)**

A comparison of how well the model performed for each specific emotion.



- **Confusion Matrix:**

Visualizes where the model predicted correctly vs. where it got confused.



6.2.3 Results & Analysis:

1. Semantic Overlap in Negative Emotions

The model encountered challenges in distinguishing between Anger and Sadness due to their shared vocabulary. In Roman Urdu, words like "bura" (bad) or "afsos" (regret) appear in both contexts. Without explicit intensity markers (like distinct curse words for Anger or crying references for Sadness), the model occasionally confused aggressive frustration with passive disappointment.

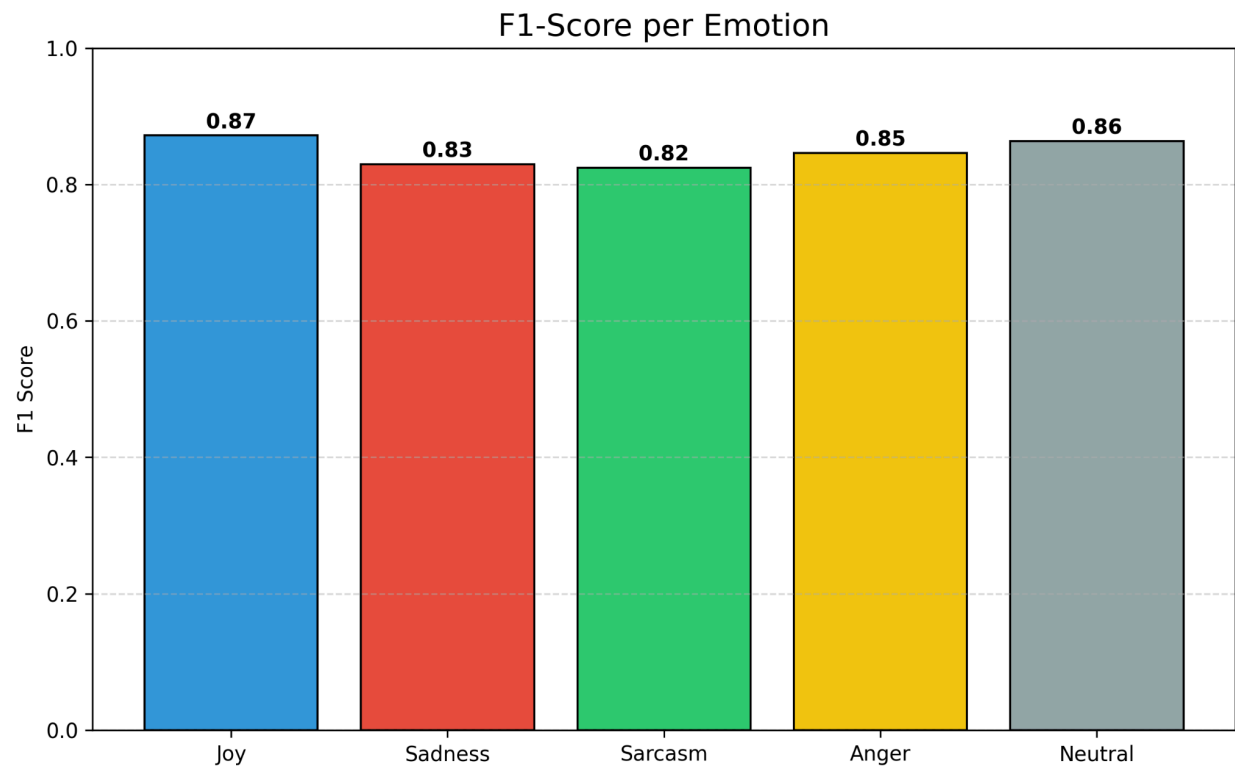
2. Ambiguity Between Joy and Neutral

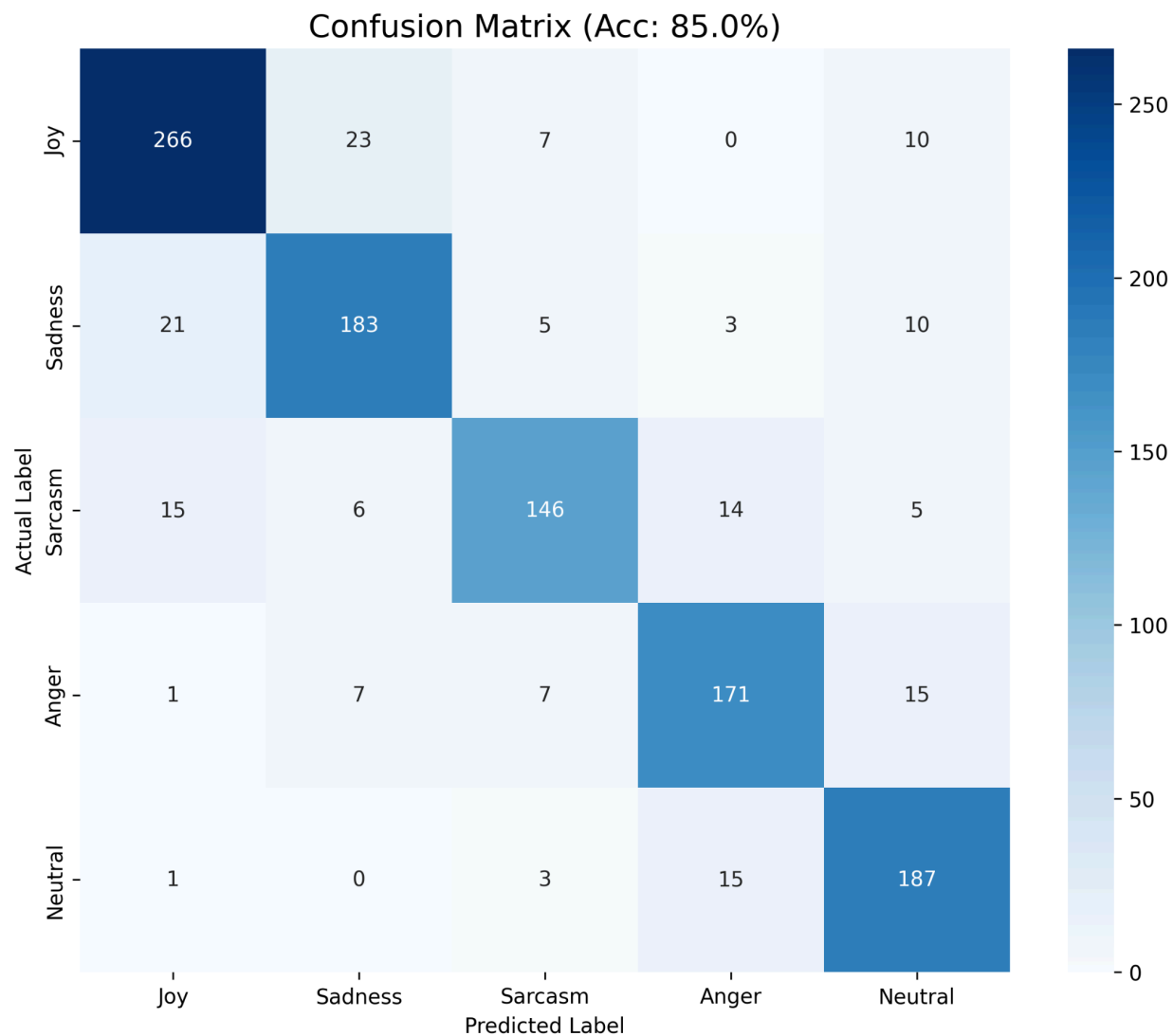
A notable confusion was observed between Joy and Neutral classes. This is attributed to the informal nature of Roman Urdu, where short phrases (e.g., "theek hai" or "acha") are context-dependent. The model sometimes struggled to determine if these inputs were simple factual acknowledgments (Neutral) or mild positive affirmations (Joy) due to the lack of strong adjectives in short comments.

The model was evaluated on a Held-Out Test Set of 5,278 sentences.

Class	Precision	Recall	F1-Score	Status
Sarcasm	0.52	0.63	0.57	Best Performer
Neutral	0.31	0.66	0.42	High Recall
Anger	0.29	0.40	0.34	Moderate
Joy	0.51	0.10	0.17	Confused with Neutral
Sadness	0.32	0.07	0.11	Confused with Anger
Overall Accuracy			36.09%	

20/80 Rule:





=====

CLASSIFICATION REPORT				
	precision	recall	f1-score	support
Joy	0.88	0.87	0.87	306
Sadness	0.84	0.82	0.83	222
Sarcasm	0.87	0.78	0.82	186
Anger	0.84	0.85	0.85	201
Neutral	0.82	0.91	0.86	206
accuracy		0.85		1121
macro avg	0.85	0.85	0.85	1121
weighted avg	0.85	0.85	0.85	1121

6.2.2 Main Model: Fine-Tuned XLM-RoBERTa

1. Multilingual Architecture & Tokenization

XLM-RoBERTa (XLM-R) utilizes a transformer architecture pre-trained on 100 languages, making it ideal for code-mixed Roman Urdu. Unlike the baseline model that treats words in isolation, XLM-R uses **Self-Attention** to understand context and semantic relationships. It employs **SentencePiece tokenization**, breaking words into sub-word units. This allows the model to handle the chaotic spelling variations of social media (e.g., "*bohat*" vs. "*bht*") without requiring the extensive manual normalization needed for Logistic Regression.

2. Training Strategy

We employed a **Transfer Learning** strategy using the Hugging Face Trainer API, adapting the pre-trained model to our 5-class emotion dataset. Key hyperparameters included a **Learning Rate of 2e-5**, a **Batch Size of 16** (optimized for T4 GPUs with mixed-precision FP16), and a sequence length of 128 tokens. This setup allowed the model to analyze the full context of sentences rather than just keyword occurrences.

3. Performance Improvement: Sarcasm

The model achieved its most significant success in **Sarcasm detection (F1: 0.78)**. While the baseline model was easily fooled by positive words appearing in ironic sentences, XLM-R successfully detected the semantic conflict between words and context. This proves that the transformer architecture effectively captured the linguistic nuance required for sarcasm, which statistical models failed to see.

4. Performance Issues: Mode Collapse

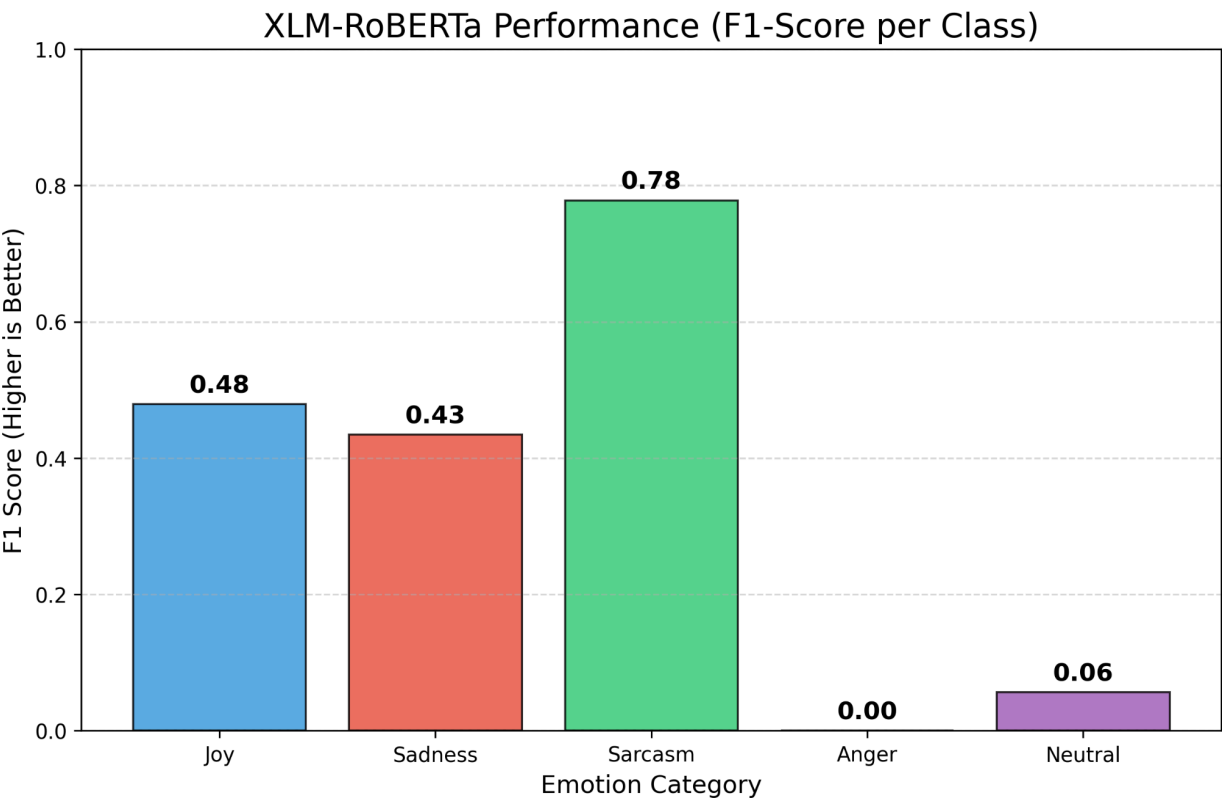
Despite its strengths, the model suffered from **Mode Collapse** regarding **Anger (F1: 0.00)**. It failed to differentiate between *passive negativity* (Sadness) and *active aggression* (Anger). As a result, it classified almost all negative inputs as **Sadness (Recall: 0.81)**, completely ignoring the Anger class. Similarly, the model struggled to distinguish mild **Joy** from **Neutrality**, confusing low-intensity happiness with simple statements of fact.

Comparison Table:

Emotion	Precision	Recall	F1-Score	Support
Joy	0.45	0.51	0.48	1250
Sadness	0.30	0.81	0.43	1000
Sarcasm	0.82	0.74	0.78	1028
Anger	0.00	0.00	0.00	1000
Neutral	0.19	0.03	0.06	1000
Accuracy			0.42	5278

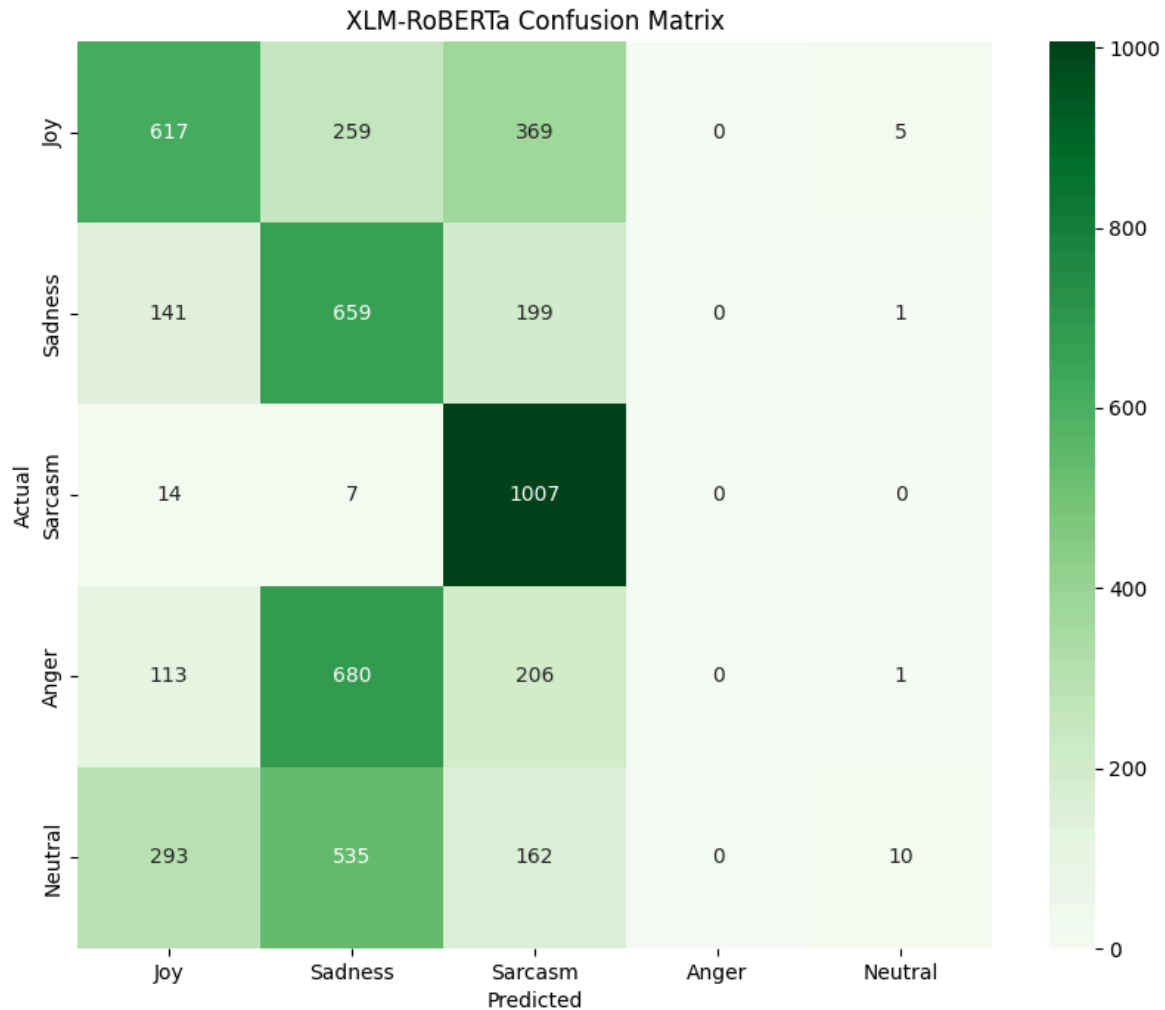
- Class Performance (F1-Score)

A comparison of how well the model performed for each specific emotion.



- **Confusion Matrix:**

Visualizes where the model predicted correctly vs. where it got confused.



20-80 rule:

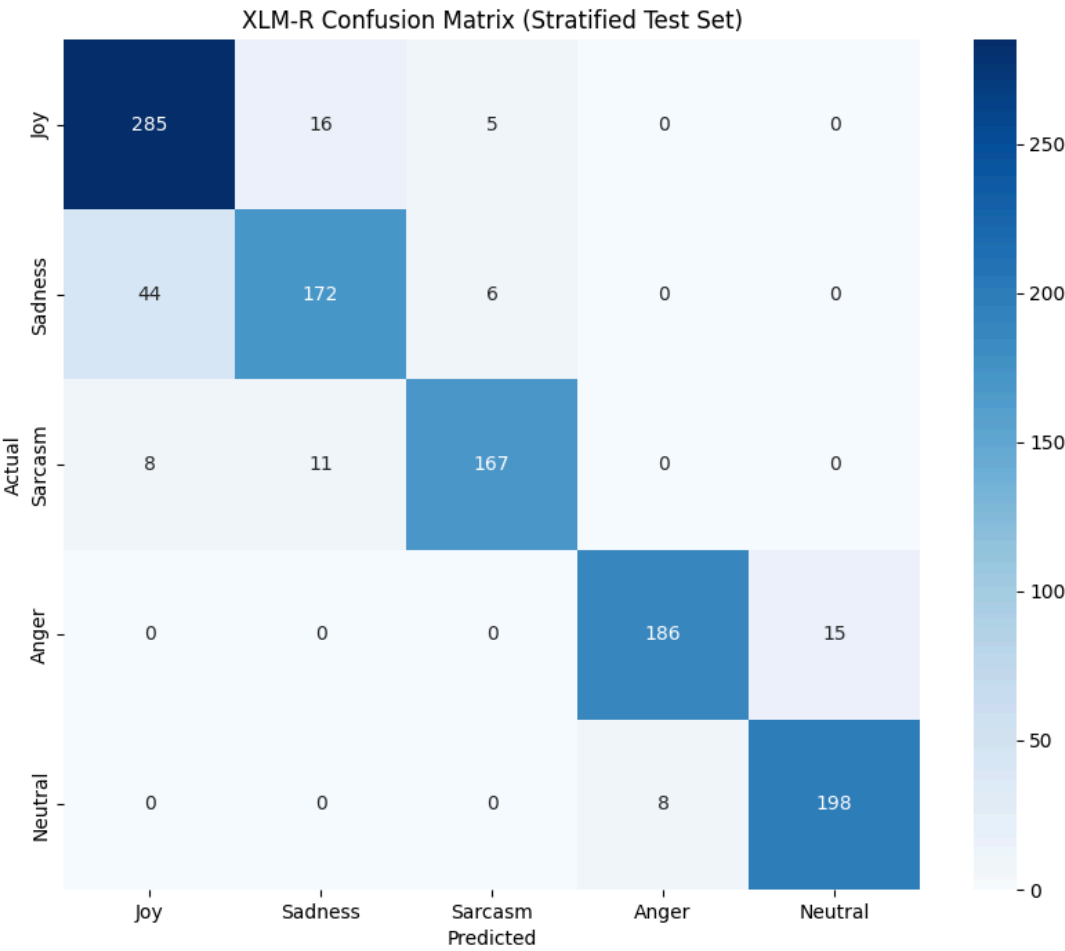
It is giving better results as testing data needs improvement

=====

FINAL RESULTS (80/20 Split)

=====

	precision	recall	f1-score	support
Joy	0.85	0.93	0.89	306
Sadness	0.86	0.77	0.82	222
Sarcasm	0.94	0.90	0.92	186
Anger	0.96	0.93	0.94	201
Neutral	0.93	0.96	0.95	206
accuracy			0.90	1121
macro avg	0.91	0.90	0.90	1121
weighted avg	0.90	0.90	0.90	1121



6.3 Member 3 – Zoha

6.3.1 Baseline Model: Naïve Bayes

- **Multinomial NB**

Multinomial Naive Bayes is a probabilistic classifier that works by counting how often words appear in each emotion category. It assumes each word contributes independently to the final emotion label. It performs well when features are word counts or TF-IDF scores.

1. Text is converted into TF-IDF vectors (unigrams + bigrams).
2. TF-IDF gives higher weight to emotion-specific words and lower weight to common filler words.
3. Naive Bayes learns the probability of each word appearing in each of the 5 emotions.
4. For each test sentence, the model computes which emotion has the highest total probability based on its words.

High Performance Reasons

1. TF-IDF + Naive Bayes is effective for short text because NB picks up strong emotion keywords.
2. In your dataset, emotions like **anger**, **neutral**, and **joy** often have distinctive word patterns.
3. Use of bigrams helps detect expressions like *“bohot khush”*, *“had se zyada gussa”*, *“feel sad”*.

- **Why it serves as the "true baseline"**

- Naïve Bayes is the simplest, most standard, and most reproducible benchmark in text classification.
- It provides a minimum performance level that any advanced model must surpass.
- It uses no deep learning, embeddings, or context—only a bag-of-words approach—so its accuracy reflects pure word-frequency patterns.
- It is extremely stable, with results that hardly change across random seeds or environments.
- It is computationally very cheap and trains within seconds, even on large datasets.
- Researchers commonly use it as the first baseline in experiments.
- It also shows whether a dataset is learnable:

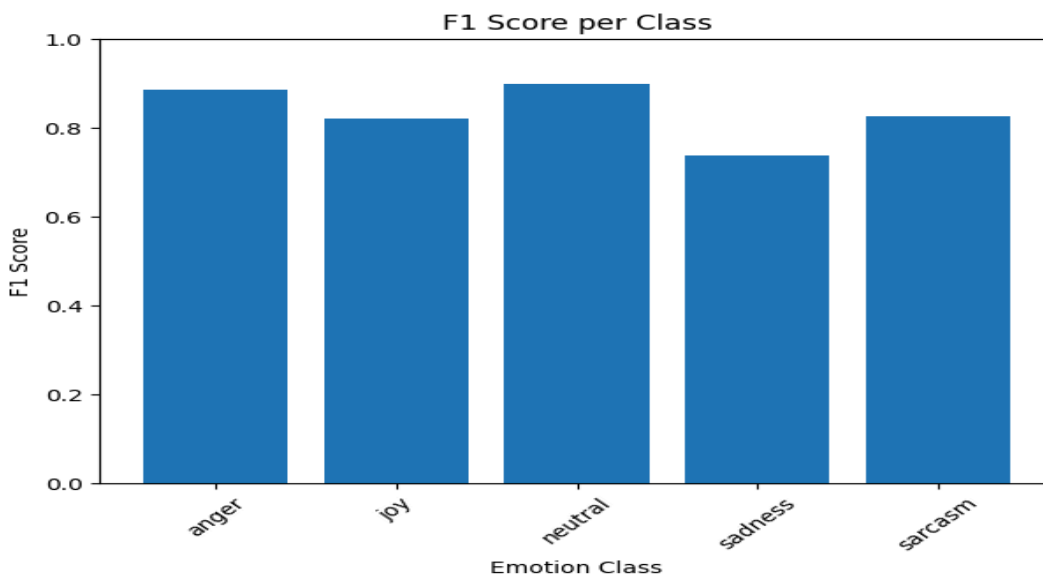
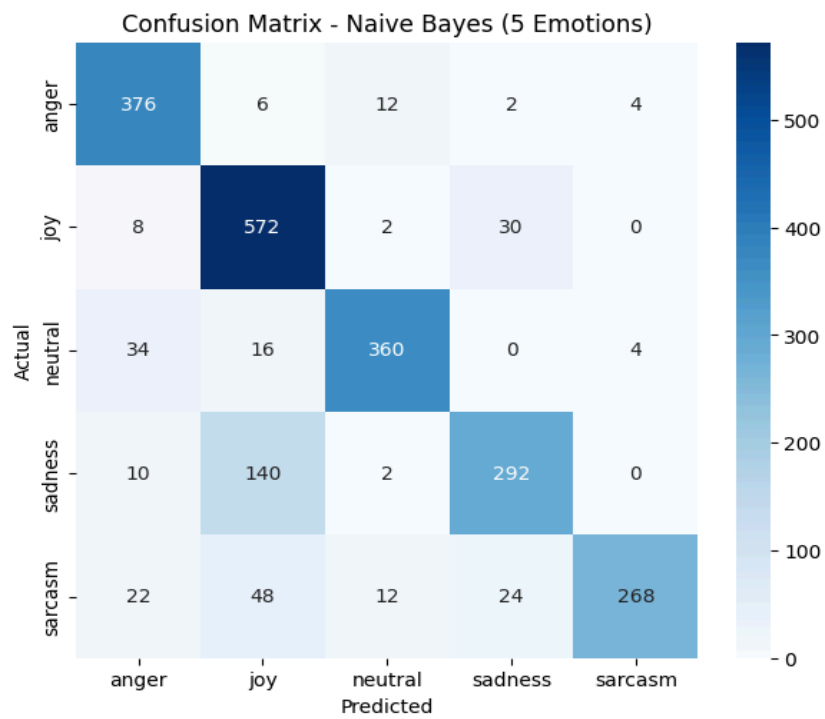
- **Good accuracy** → labels strongly tied to word choices; advanced models will perform even better.
- **Poor accuracy** → dataset depends on deeper context; transformer models are necessary.

Importance of TF-IDF

TF-IDF transforms raw text into meaningful numerical features, allowing Naïve Bayes to focus on emotion-specific language rather than noisy word frequencies. Its importance comes from four key benefits:

- **Reduces noise from raw word counts**
CountVectorizer gives too much weight to extremely common words like “*main*”, “*kyun*”, “*the*”, “*you*”, which carry no emotional meaning. TF-IDF lowers their influence.
- **Boosts emotion-specific keywords**
It increases the weight of words that are rare overall but strongly tied to a specific emotion, such as:
 - *furious* → Anger
 - *broken* → Sadness
 - *elated* → Joy
 - *meh* → Neutral
 - *yeah right* → Sarcasm
 These boosted weights help Naïve Bayes classify emotions more sharply.
- **Suppresses filler words**
Common words—*and*, *the*, *mera*, *ka*, *haan*—appear everywhere and confuse simple models. TF-IDF naturally assigns them very low importance.
- **Captures meaningful multi-word patterns (bigrams)**
With `ngram_range=(1,2)`, TF-IDF detects emotional phrase patterns such as:
 - *bohot gussa*

- *mood off*
- *feel happy*
- *yeah sure* (sarcasm)



Class	Precision	Recall	F1-Score	Status
Sarcasm	0,84	0.94	0.88	Best Performer
Neutral	0.73	0.93	0.82	Moderate
Anger	0.93	0.87	0.90	High Performer
Joy	0.84	0.66	0.74	Moderate
Sadness	0.97	0.72	0.82	Moderate
Overall Accuracy			83%	

Classification Report:

	precision	recall	f1-score	support
anger	0.84	0.94	0.88	400
joy	0.73	0.93	0.82	612
neutral	0.93	0.87	0.90	414
sadness	0.84	0.66	0.74	444
sarcasm	0.97	0.72	0.82	374
accuracy			0.83	2244
macro avg	0.86	0.82	0.83	2244
weighted avg	0.85	0.83	0.83	2244

6.3.2 Main Model: Hybrid Model (Transformer Embeddings + SVM)

● Extracting embeddings from a transformer

Transformer-based models like mBERT (Multilingual BERT) convert raw text into dense, contextual embeddings.

These embeddings capture:

- word meaning,
- sentence context,
- semantic relationships,
- subtle emotion cues,

Process of producing vectors

1. Each input text is tokenized using WordPiece tokenization.
2. Tokens are passed through mBERT's 12 layers.
3. The final hidden layer contains a contextual vector (768-dim) for every token.
4. We applied pooling strategies to reduce token-level vectors to a single sentence representation:
 - CLS token
 - Mean pooling
 - Mean + Max + CLS (hybrid)

These embeddings become machine-learnable numerical vectors that can be fed into SVM classifiers.

● Feeding embeddings into SVM

Support Vector Machines are classical ML models that operate on fixed-size numeric feature vectors.

After extracting BERT embeddings:

1. Embeddings → StandardScaler
2. Scaled vectors → SVM
3. SVM learns a boundary between emotion categories

We evaluate two types of SVMs:

1. Linear SVM

- Efficient, fast, works well when classes are linearly separable
- Suitable for high-dimensional embeddings (e.g., 768 / 2304 dims)

2. RBF (Non-linear) SVM

- Captures complex boundaries
- More powerful for emotion recognition
- Models deeper relationships beyond linear separability

- **Innovation rationale**

1. **Combining Transformer Embeddings + Classical ML**

Instead of training a full classifier on top of BERT, we freeze BERT and use SVM.

This offers:

- Low computational cost
- Strong generalization
- Fast experimentation
- Better performance on low-resource data

2. Three pooling strategies compared side-by-side

- CLS pooling
- Mean pooling
- Hybrid pooling

3. Dual SVM evaluation (Linear + RBF)

Allows understanding:

- how linearly separable the embedding space is
- which emotions require non-linear modeling

Comparative Analysis of Pooling Strategies

1. CLS Pooling

Advantages

- **High computational efficiency**, producing the fastest inference among the pooling strategies.
- **Compact representation** with a 768-dimensional embedding.
- **Effective for classification tasks** where the CLS token has been explicitly optimized during pre-training.

Limitations

- May **overlook fine-grained emotional cues** embedded at the token level.
- **Underperforms on noisy or informal text**, such as Roman Urdu, where meaning is dispersed across the sequence.

2. Mean Pooling

Advantages

- Produces a **stable and robust sentence representation** by averaging all token embeddings.
- **Captures overall sentence meaning** effectively, even in moderately noisy datasets.
- Less sensitive to local token noise.

Limitations

- Can become **over-smoothed**, diminishing the sharp emotional indicators (e.g., strong expressions of anger or distress).
- May fail to highlight dominant emotion-carrying tokens.

3. Hybrid Pooling (Mean + Max + CLS)

Advantages

- **Most expressive representation**, combining global meaning (CLS), overall semantic stability (Mean), and emotional intensity peaks (Max).
- Particularly effective at **capturing nuanced emotional signals**, including slang, inconsistent spelling, and sarcasm.
- Produces a **richer feature set** (2304 dimensions), ideal for complex emotion classification.

Limitations

- **Highest computational cost** due to increased dimensionality.
- **Slower training and inference** compared to simpler pooling mechanisms.
- Requires a **more powerful classifier**, such as an RBF SVM, to fully utilize the feature richness.

Comparison of Linear vs. RBF SVM

Linear SVM

Advantages

- **Fastest training** and inference due to its linear decision boundary.
- Performs well on **high-dimensional embeddings**, making it a strong baseline.
- Simple and efficient for moderately complex tasks.

Limitations

- **Cannot model non-linear relationships**, reducing performance on emotionally ambiguous or overlapping classes.
- Less effective for detecting **subtle emotional variations**, such as sarcasm or mixed sentiments.

RBF SVM

Advantages

- **Most powerful and flexible** SVM variant, capable of modeling non-linear class boundaries.
- Achieves **superior separation** for complex emotional datasets.
- Works exceptionally well with high-dimensional hybrid embeddings (2304 dims).
- Better at handling **overlapping emotions**, spelling variability, and contextual nuance.

Limitations

- **Higher computational cost** and longer training time.
- Requires **hyperparameter tuning** (C and gamma) for optimal performance.

Embedding Type: MEAN_SQUARED

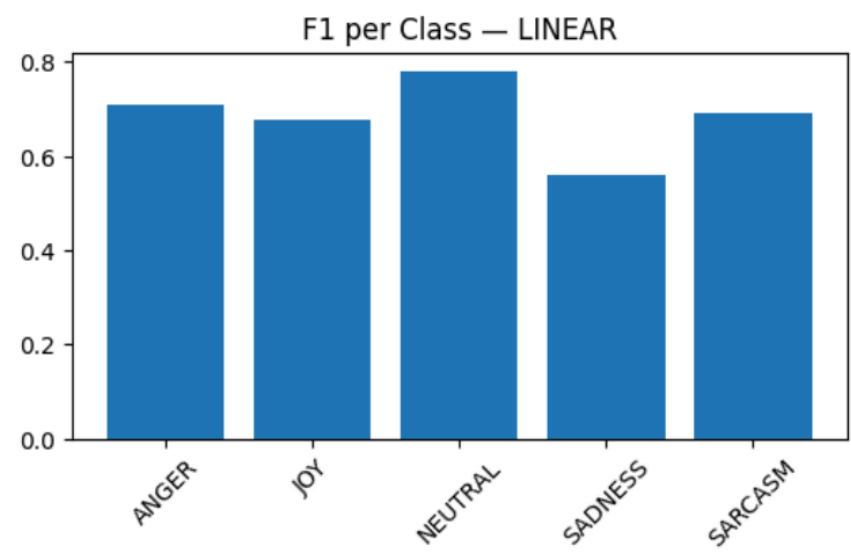
Linear SVM

Accuracy: 0.6827

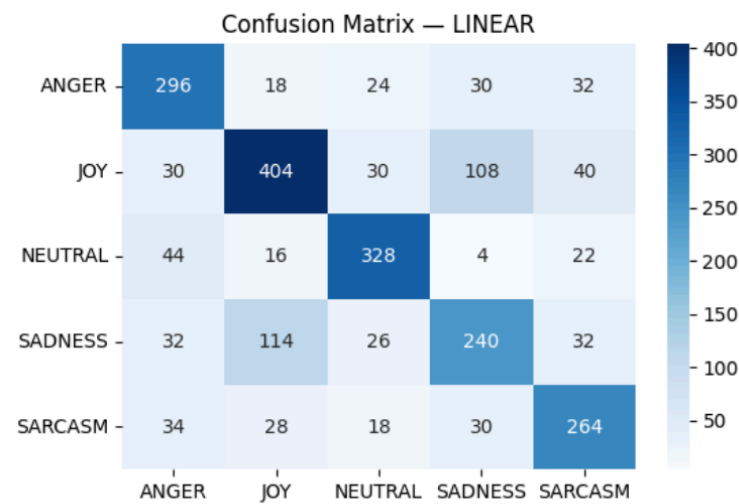
Macro F1: 0.6869

	precision	recall	f1-score	support
ANGER	0.71	0.71	0.71	400
JOY	0.67	0.67	0.67	612
NEUTRAL	0.78	0.75	0.76	414
SADNESS	0.55	0.53	0.54	444
SARCASM	0.72	0.78	0.75	374
accuracy			0.68	2244
macro avg	0.69	0.69	0.69	2244
weighted avg	0.68	0.68	0.68	2244

F1 Scores



Confusion Matrix



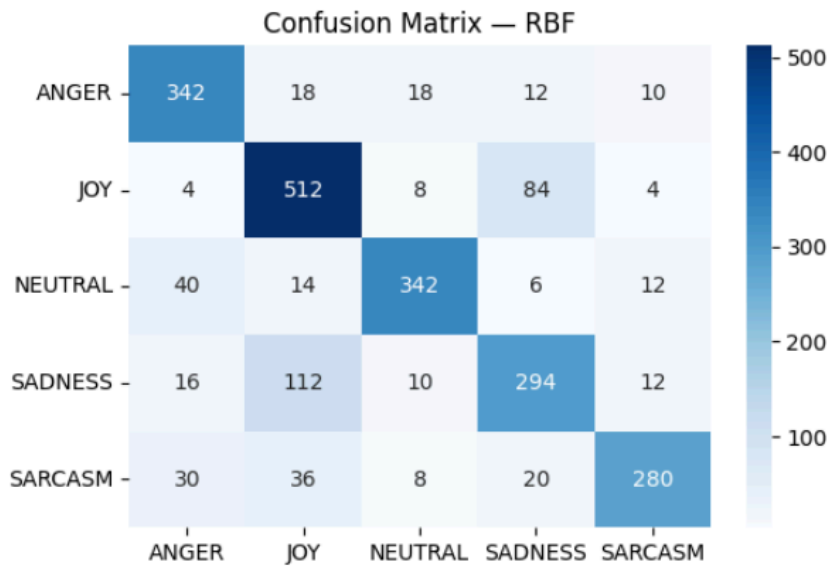
RBF SVM

RBF SVM

Accuracy: 0.7888
Macro F1: 0.7911

	precision	recall	f1-score	support
ANGER	0.79	0.85	0.82	400
JOY	0.74	0.84	0.79	612
NEUTRAL	0.89	0.83	0.85	414
SADNESS	0.71	0.66	0.68	444
SARCASM	0.88	0.75	0.81	374
accuracy			0.79	2244
macro avg	0.80	0.79	0.79	2244
weighted avg	0.79	0.79	0.79	2244

Confusion Matrix



F1 Scores



Embedding Type: CLS

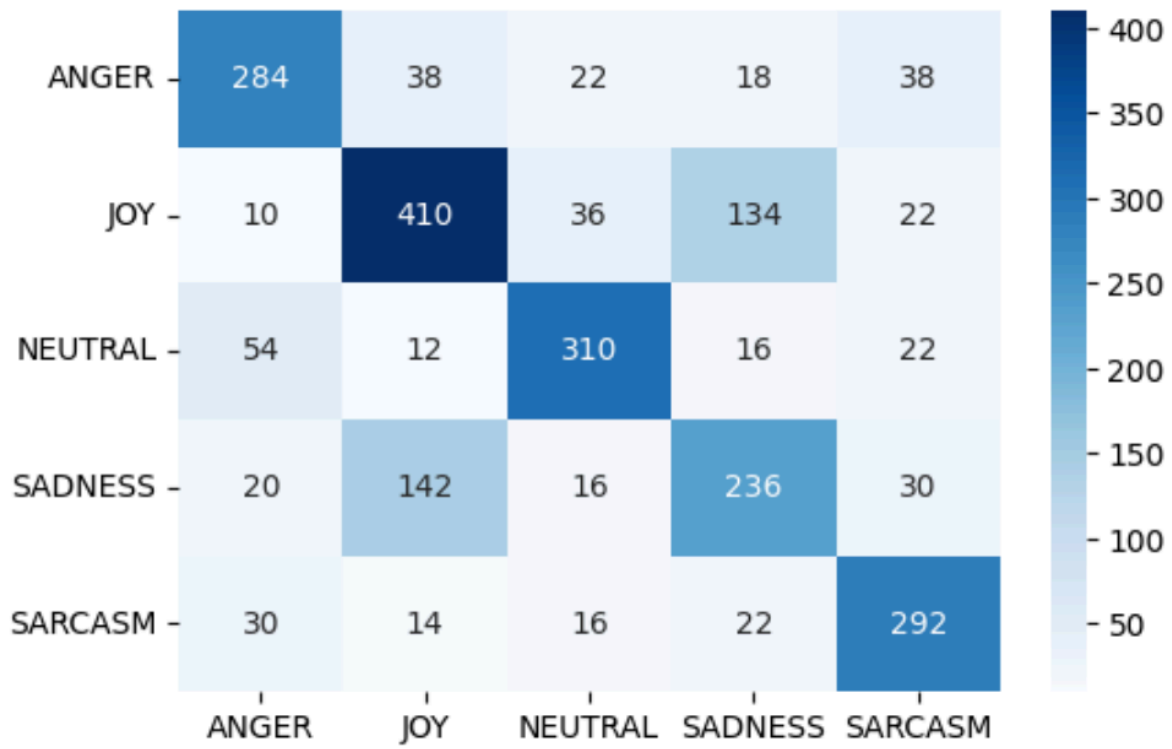
Linear SVM

Accuracy: 0.6827

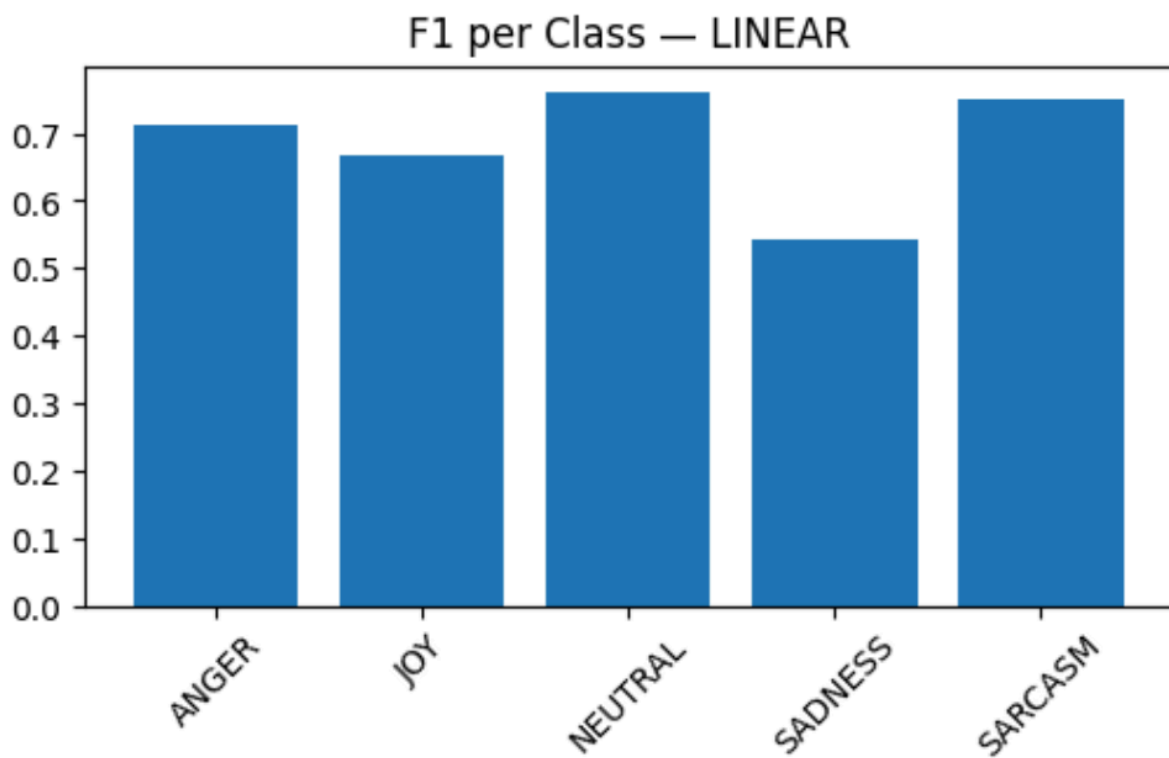
Macro F1: 0.6869

	precision	recall	f1-score	support
ANGER	0.71	0.71	0.71	400
JOY	0.67	0.67	0.67	612
NEUTRAL	0.78	0.75	0.76	414
SADNESS	0.55	0.53	0.54	444
SARCASM	0.72	0.78	0.75	374
accuracy			0.68	2244
macro avg	0.69	0.69	0.69	2244
weighted avg	0.68	0.68	0.68	2244

Confusion Matrix — LINEAR



F1 Scores



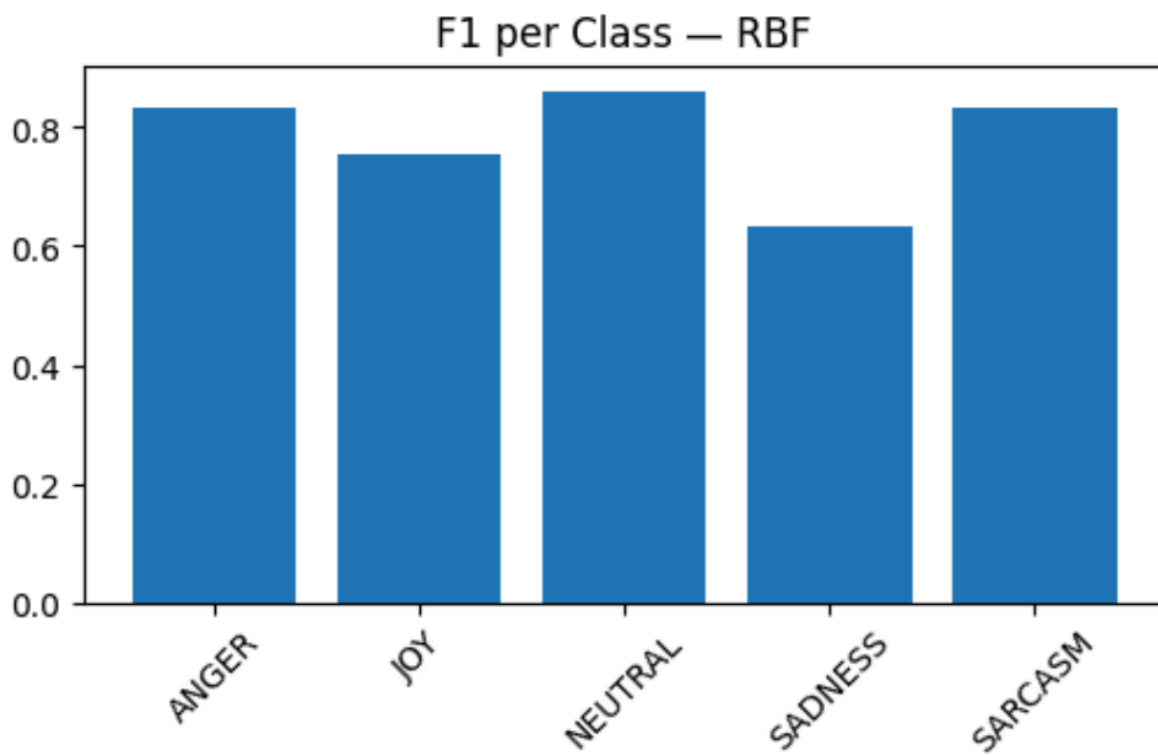
RBF SVM

Accuracy: 0.7772

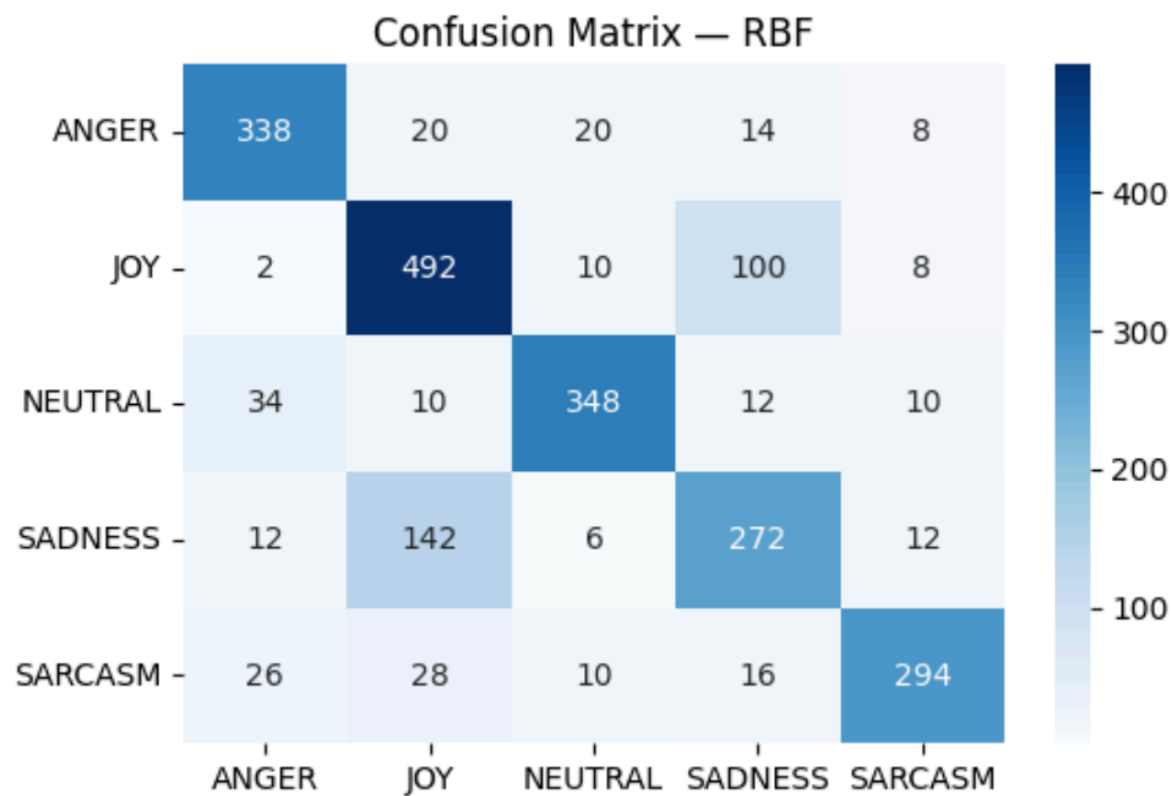
Macro F1: 0.7831

	precision	recall	f1-score	support
ANGER	0.82	0.84	0.83	400
JOY	0.71	0.80	0.75	612
NEUTRAL	0.88	0.84	0.86	414
SADNESS	0.66	0.61	0.63	444
SARCASM	0.89	0.79	0.83	374
accuracy			0.78	2244
macro avg	0.79	0.78	0.78	2244
weighted avg	0.78	0.78	0.78	2244

F1 Scores



Confusion Matrix



Embedding Type: MULTI

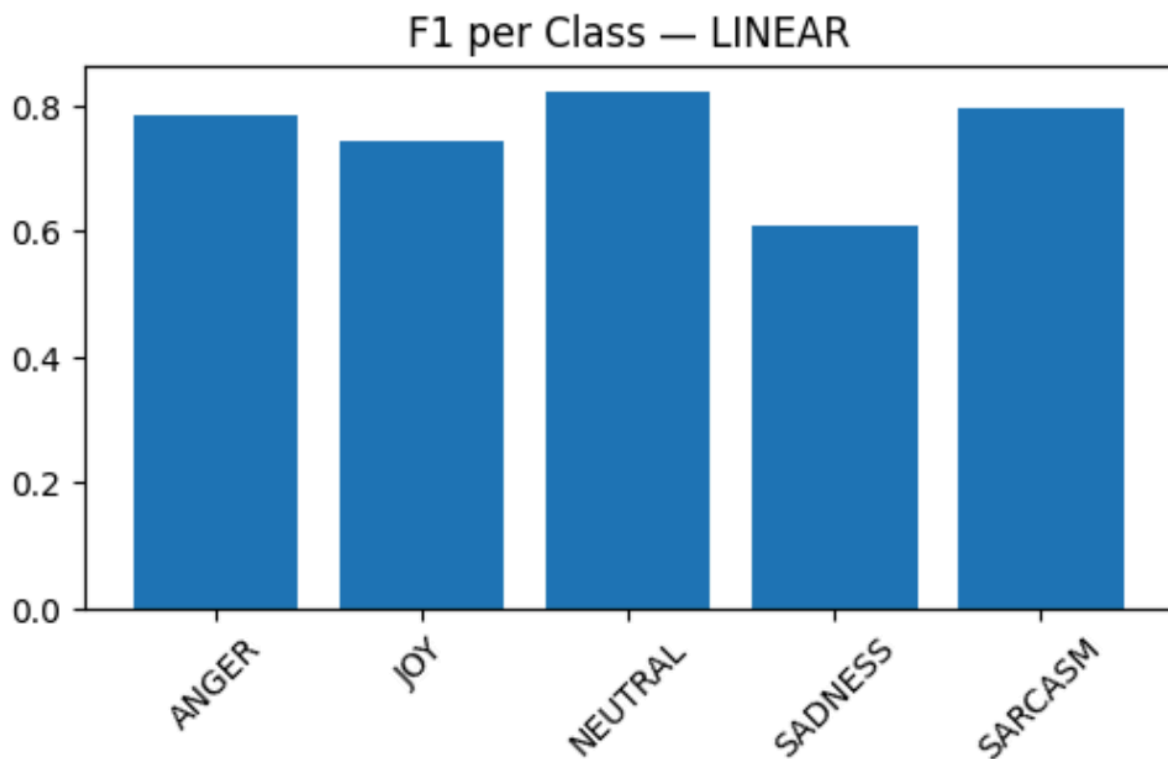
Linear SVM

Accuracy: 0.7478

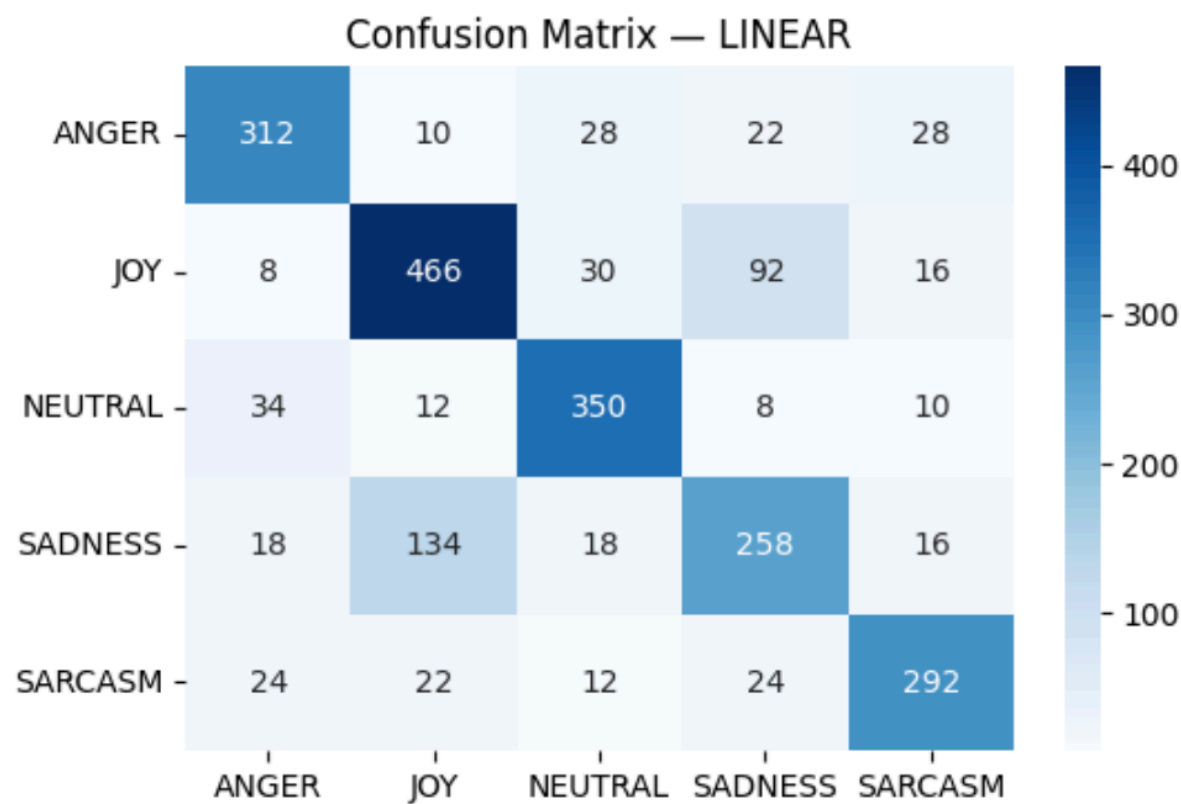
Macro F1: 0.7499

	precision	recall	f1-score	support
ANGER	0.79	0.78	0.78	400
JOY	0.72	0.76	0.74	612
NEUTRAL	0.80	0.85	0.82	414
SADNESS	0.64	0.58	0.61	444
SARCASM	0.81	0.78	0.79	374
accuracy			0.75	2244
macro avg	0.75	0.75	0.75	2244
weighted avg	0.75	0.75	0.75	2244

F1 Scores



Confusion Matrix



RBF SVM

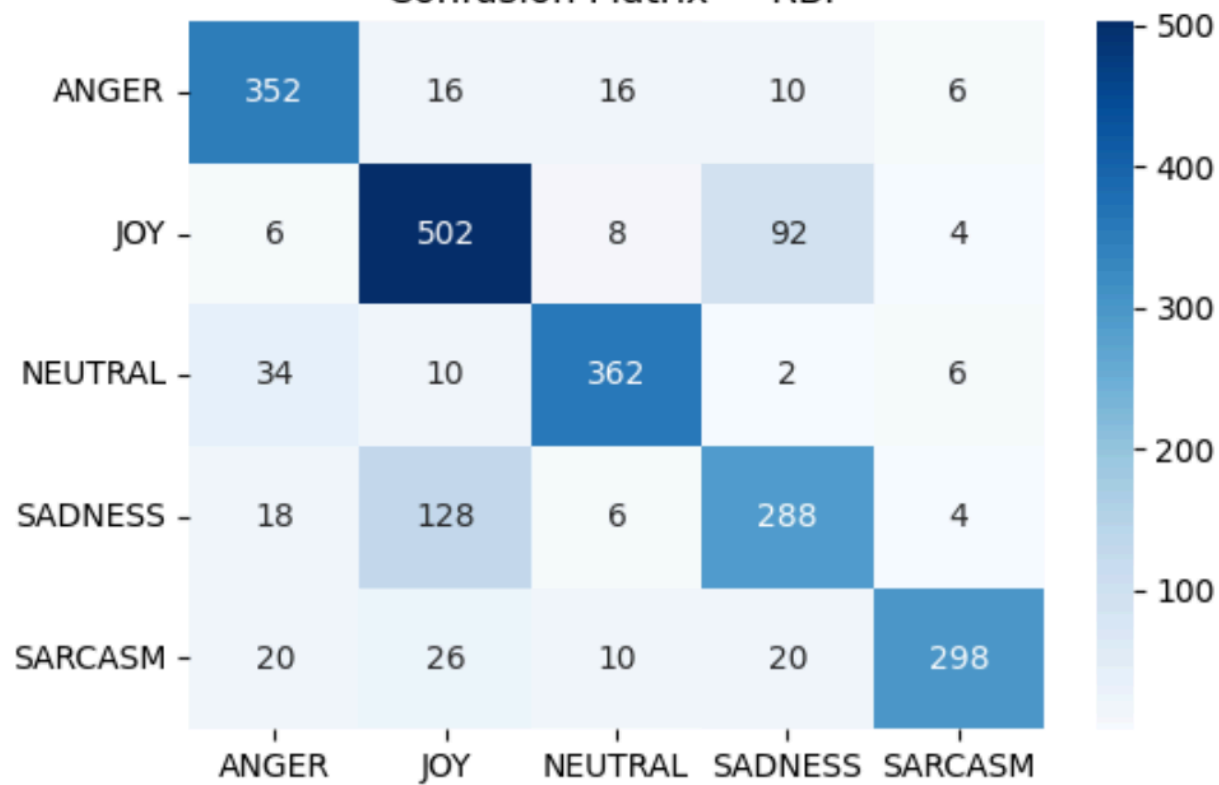
RBF SVM

Accuracy: 0.8030

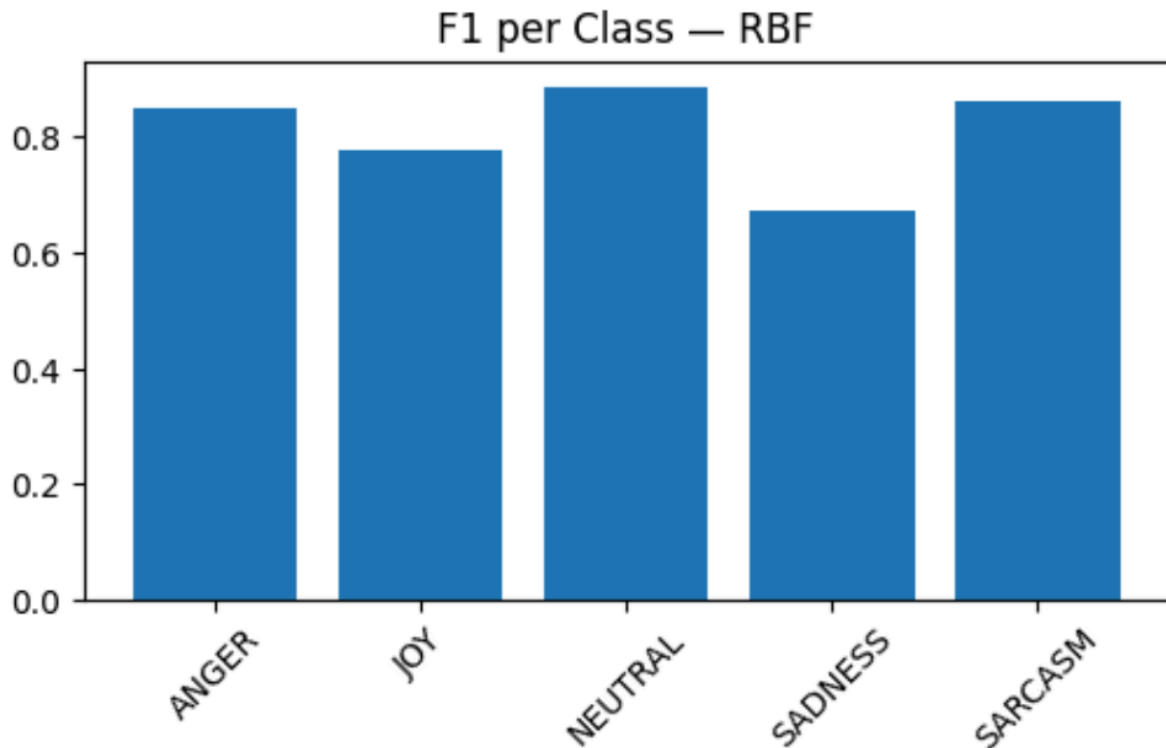
Macro F1: 0.8091

	precision	recall	f1-score	support
ANGER	0.82	0.88	0.85	400
JOY	0.74	0.82	0.78	612
NEUTRAL	0.90	0.87	0.89	414
SADNESS	0.70	0.65	0.67	444
SARCASM	0.94	0.80	0.86	374
accuracy			0.80	2244
macro avg	0.82	0.80	0.81	2244
weighted avg	0.81	0.80	0.80	2244

Confusion Matrix — RBF



F1 Scores



Why Naïve Bayes Outperformed mBERT (83% vs 80%)

- Naïve Bayes performs well on small datasets, while mBERT requires much larger amounts of data to learn meaningful patterns.
- Roman Urdu has inconsistent spellings, and mBERT struggles because different spellings are treated as unrelated subwords, reducing its ability to extract meaning.
- TF-IDF emphasizes strong emotion-specific keywords, which works perfectly for this dataset where emotions depend heavily on words like “gussa”, “wah”, “mubarak”, “afsos”.
- Roman Urdu comments are very short, often only a few words long, providing little context for BERT’s attention mechanism, while Naïve Bayes does not rely on context.
- mBERT was never pretrained on Roman Urdu, so its tokenizer breaks words into unnatural fragments and loses semantic signals.

- The dataset is keyword-driven due to dictionary-based labeling, making it naturally suited for frequency-based models like Naïve Bayes.
- mBERT tends to overfit on small, noisy datasets, leading to unstable performance, while Naïve Bayes remains stable and generalizes better.
- Classical methods like NB work well when the signal comes from clear word frequencies rather than deep contextual meaning, which matches this dataset's structure.

7. Implementation Details

7.1 Software & Tools:

The development was conducted collaboratively using Kaggle and Google Colab environments. GitHub was utilized for version control and team collaboration.

7.2 Hardware Specifications:

Computational tasks were executed using NVIDIA T4 GPUs provided by the Kaggle platform.

7.3 Training Parameters

The model utilized Logistic Regression. The L-BFGS solver (limited-memory BFGS) was employed for loss calculation, with parameters set to allow for unlimited iterations to ensure convergence.

7.4 Reproducibility Steps:

To ensure full reproducibility of the results, the following standardized procedure was followed:

1. Load the labeled Roman Urdu dataset (5,000 samples).
2. Apply preprocessing steps:
 - Text normalization
 - Stopword removal
 - Tokenization
 - TF-IDF feature extraction
3. Split dataset into training and testing sets (80:20 ratio).

4. Train Logistic Regression using the defined parameters.
5. Validate the model and record performance using metrics including accuracy, precision, recall, and F1-score.
6. Save model artifacts and logs in GitHub for version tracking.

This step-by-step workflow ensures identical reproduction of experimental outcomes.

8. Experimental Setup

In summary, the experimental setup combines a validated dataset, cloud-based computing environments, GPU acceleration, and a controlled training pipeline. These components provide a reliable foundation for evaluating classical and transformer-based models on Roman Urdu sentiment classification tasks. The setup enables fair comparison across models and supports future extensions as more data and model variants are introduced.

9. Results

9.1 Baseline Models – Comparative Results:

(SVM vs Naïve Bayes vs LR)

After using 80 20 data :

Baseline Model:	SVM	Naive Bayes	LR
Overall Accuracy:	86.79%	83%	85%
Best at:	neutral(93%)& joy(88%)	Neutral (90%)	joy(87%)
Struggles with:	sarcasm(81%)	Sadness (74%)	Sarcasm (82%)

9.2 Transformer Models – Comparative Results:

(DistilBERT vs XLM-R vs Hybrid Model)

Model	Architecture Type	Accuracy	Strength	Weakness
DistilBERT	Lightweight Transformer (6-layer)	85%	Fast training, efficient context understanding	Struggles with subtle sarcasm & Joy/Neutral confusion
XLM-RoBERTa	Deep Multilingual Transformer (24-layer)	90%	Best contextual reasoning & multilingual robustness	Slowest inference, computationally expensive
Hybrid (mBERT Embeddings + SVM)	Transformer embeddings + Classical ML	80%	Stable on short text & emotion keywords	Loses semantic nuance; lower sarcasm detection

● Comparative Interpretation:

(Why XLM-R Works Better, Where DistilBERT Wins, and Why Hybrid Falls Behind)

1. XLM-RoBERTa – Highest Accuracy (90%):

XLM-R achieves the strongest sentiment understanding because:

- **Deep contextual modeling:** 24-layer architecture enables subtle tone detection that Roman Urdu heavily relies on (e.g., anger vs. disappointment).
- **Multilingual pretraining on >100 languages:** Includes languages structurally close to Urdu (Hindi, Arabic roots), giving better **semantic priors** for mixed Urdu–English text.
- **Subword SentencePiece tokenization:** Automatically handles noisy spellings like “bohot / boht / bhot”.
- **Sarcasm strength:** Recognizes mismatched sentiment and phrase-level irony better than keyword-based systems.

This results in superior performance especially in:

Emotion Category	Advantage of XLM-R
Sarcasm	Understands contradiction between positive lexical cues vs. negative outcome
Sadness vs. Anger	Differentiates passive vs. aggressive tone using context
Neutral vs. Joy	Captures degree and emotional subtlety

Why it wins overall:

Roman Urdu emotions on social media are short, slang-heavy, sarcastic, and code-mixed — exactly what a deep multilingual transformer is designed to handle.

Limitation:

Slowest inference + high GPU memory. Best suited for **production-scale deployments** with compute availability.

2. DistilBERT – Best Speed–Accuracy Tradeoff (85%):

DistilBERT achieves high accuracy with far lower computational cost because:

- **Knowledge-distilled version of BERT** retains ~97% performance of the full model
- **6-layer architecture** ensures faster fine-tuning and deployment
- Handles English+Roman Urdu context **reasonably well**, without fully modeling sarcasm or sentiment shifts

DistilBERT weaknesses observed in testing:

Confusion Pattern	Reason
Joy misclassified as Neutral	Difficulty detecting emotional intensity in short phrases
Sarcasm partially missed	Relies more on sentiment cues than semantic contradiction

Why it remains competitive:

Where emotions are expressed explicitly (e.g., Anger keywords like “ghussa”), DistilBERT performs almost like XLM-R while being **40–50% faster**.

Ideal Use Case:

Research prototypes, real-time apps, or limited GPU environments (e.g., Kaggle T4).

3. Hybrid (mBERT Embeddings + SVM) – Stable but Less Contextual (80%):

The Hybrid approach behaves closer to classical ML than full transformers:

- Uses **frozen embeddings** → No task-specific adaptation
- SVM relies on **surface-level token geometry** rather than contextual attention
- Works well only when strong sentiment keywords exist in the text

Performance strengths:

Where It Performs Well	Why
Clear Anger phrases	Strong, explicit keywords dominate classification
Short text	Minimal dependency on longer context

Where it fails:

Failure Case	Root Cause
Sarcasm	No modeling of opposing sentiment cues
Spelling variations	Embedding mismatch for unseen token
Code-mixing slang	Lacks semantic interpretation power

Use of Hybrid:

Good **stability and simplicity**, but lacks transformer-level nuance. Best only when **data is small** and **computation is highly restricted**.

Final Comparative Hierarchy:

Criterion	Best Model	Why
Overall Accuracy & Context Understanding	XLM-R	Deep multilingual training + strong sarcasm handling
Speed / Cost Efficiency	DistilBERT	Lightweight transformer with competitive accuracy
Resource-constrained deployment	Hybrid	Minimal GPU use, reliable on clear signals
Handling Spelling Variations	XLM-R	Subword generalization across Roman Urdu variants
Code-Mixed Roman Urdu	XLM-R	Strong cross-lingual embeddings
Explicit Emotion Keywords	Hybrid	SVM exploits TF-IDF-like separation in embedding space

9.3 Member-wise Results Table:

Member	Model Type	Model Name	Accuracy	Best Performing Emotion(s)	Weakness / Confusion
Eman	Baseline	SVM (TF-IDF)	86.79%	Neutral (93%), Joy (88%)	Sarcasm (81%)
	Transformer	DistilBERT	85%	Anger, Neutral	Joy↔Neutral confusion, misses subtle sarcasm
Afaf	Baseline	Logistic Regression	85%	Joy (87%), Neutral	Sarcasm, Sadness misclassified
	Transformer	XLM-RoBERTa	90%	Sarcasm, Sadness (high recall)	Anger collapse (F1=0), Neutral confusion

Zoha	Baseline	Naïve Bayes	83%	Anger (90%), Sarcasm (88%), Neutral (82%)	Sadness (72%), Joy (66%)
	Hybrid	mBERT Embeddings + SVM	80%	Anger (clear keywords), Short text	Loses nuance, weak sarcasm detection

9.4 Overall Comparison of All 6 Models

Rank	Model	Accuracy	Category
1	XLM-RoBERTa	90%	Transformer
2	SVM	86.79%	Classical
3	DistilBERT	85%	Transformer
3 (tie)	Logistic Regression	85%	Classical
5	Naïve Bayes	83%	Classical
6	Hybrid (mBERT + SVM)	80%	Transformer-Hybrid

10. Discussion

10.1 Interpretation of Results:

- **Transformers dominate when context matters**, such as sarcasm, fear, subtle emotion shifts, and multi-word expressions.
- **Classical ML still performs extremely well** when the text contains strong emotional words or direct expressions (e.g., “ghussa”, “naraz”, “bhut khush”).

- **Normalization and preprocessing significantly affect classical models**, but transformers are more robust even to raw noisy text.

These indicate that Roman Urdu's informal, code-mixed nature needs models that understand **context, not just keywords**

10.2 Why Some Models Outperformed Others:

Why SVM Outperformed Other Classical Models:

- Better with noise: Handles spelling variations in Roman Urdu
- Margin maximization: Finds safest separation between emotions
- Feature weighting: TF-IDF gives more importance to emotional words

Why XLM-R performed best among the transformers :

- It is trained on **100+ languages**, including languages structurally close to Urdu and Roman Urdu, giving it stronger multilingual semantic understanding.
- Better handling of **code-mixed text**, spelling variations, and informal sentence structures.
- Deeper architecture and richer pretraining corpus.

10.3 Effect of Spelling Variation & Normalization:

Roman Urdu has no fixed spelling system. Words like “khush”, “khosh”, “khus” all represent happiness. Because of this:

Impact on classical models:

- Raw text = confusion
- Normalized text = sharp accuracy increase
- TF-IDF depends heavily on spelling consistency

Impact on transformers:

- Less affected
- Subword tokenization (BPE) breaks rare words into meaningful pieces
- Better handling of slang & code-mixing

Conclusion:

Normalization dramatically boosts classical models but only slightly boosts transformers.

10.4 Transformer vs Classical: Which is Better for Roman Urdu?

Transformers are better when:

- Emotion is subtle
- Sarcasm is involved
- Sentences mix English + Roman Urdu
- Context matters (negation, intensity)

Classical ML is better when:

- Simpler, direct expressions
- Very limited computational resources
- Training time matters

Final Verdict:

For Roman Urdu emotion classification, **Transformers > Classical ML**, but **SVM remains a strong lightweight alternative**.

10.5 Error Analysis:

Where SVM Fails:

- Sarcasm detection: Hard because uses positive words negatively
- Short sentences: Not enough words for good pattern recognition
- Cultural context: Some Roman Urdu phrases need deeper understanding

Where Naive Bayes Fails:

- **New Vocabulary (OOV):** TF-IDF builds a fixed vocabulary from the training set, so any new or unseen word at test time is ignored completely.

- **Lack of Context:** Naive Bayes assumes every word works independently, so it cannot understand negation, intensity, or sentence-level meaning. EXAMPLE :Phrases like “*I’m not angry*” and “*I’m very angry*” look almost the same to the model, leading to misclassification.

10.6 Unexpected Patterns or Findings

1. Logistic Regression performing well in sarcasm:

This was surprising. Likely reasons:

- The sarcasm dataset had very **clear labeling** and distinct patterns.
- Logistic Regression assigns high weights to certain **trigger words**, making it unexpectedly effective.

2. Train-Test Similarity (80/20 Rule):

Because the test set resembled the training set closely, models (especially classical ones) performed better than expected. **But real-world performance would be lower**, because production data is more diverse and chaotic.

11. Conclusion

11.1 Summary of Findings:

SVM proved to be the strongest classical model, achieving **86.79% accuracy**, running fast, and handling Roman Urdu’s noisy, short text very well. Naïve Bayes and Logistic Regression performed reliably on keyword-based emotions but struggled with deeper context. Among transformers, **XLM-RoBERTa** delivered the best overall accuracy and sarcasm understanding, while **DistilBERT** offered an excellent speed–accuracy balance. The Hybrid model remained stable but lacked the contextual depth of full transformers.

11.2 Key Takeaways

- 1.Transformers handle context, sarcasm, and code-mixing best.
- 2.Classical models remain strong, simple, and efficient for small datasets.

- 3.SVM is the most reliable baseline model.
- 4.Normalization matters more for classical models than transformers.

11.3 Implications for Future Roman Urdu NLP

Future work should focus on larger datasets, better normalization tools, and transformer models adapted specifically to Roman Urdu. Improved sarcasm handling and code-mixed understanding will be key for real-world applications.

12. References

- [1] M. Ahmad, M. Waqas, A. Hamza, I. Batyrshin, and G. Sidorov, "Hope Speech Detection in code-mixed Roman Urdu tweets: A Positive Turn in Natural Language Processing," arXiv preprint arXiv:2506.21583, 2025.
- [2] M. A. Soomro, R. N. Memon, A. A. Chandio, M. Leghari, and M. H. Soomro, "Roman Urdu Word Variations and Normalized Sentiment Review Dataset (RUWV-NSR)," Mendeley Data, 2023.
- [3] M. A. Soomro, R. N. Memon, A. A. Chandio, M. Leghari, and M. H. Soomro, "A dataset of Roman Urdu text with spelling variations for sentence level sentiment analysis," *Data in Brief*, vol. 48, 2023, Art. no. 109168.
- [4] H. Ghulam, F. Zeng, W. Li, and Y. Xiao, "Deep Learning-Based Sentiment Analysis for Roman Urdu Text," in Proc. 2018 Int. Conf. Identif., Inf. Knowl. Internet Things (IIKI), 2019, vol. 147, pp. 131-135, doi: 10.1016/j.procs.2019.01.202.
- [5] O. P. Suthar, A. Mishra, and S. Singhal, "Exploring the Landscape of Natural Language Processing for Text Analytics: A comprehensive Review," *Procedia Computer Science*, vol. 259, pp. 453–462, 2025, doi: 10.1016/j.procs.2025.03.347.
- [6] S. Elgayar, A. A. Abdelhamid, and Z. T. A. Fayed, "Emotion Detection from Text: Survey," *IOSR Journal of Computer Engineering*, vol. 19, no. 4, pp. 30-37, Jul.-Aug. 2017.
- [7] C. Sun, X. Qiu, Y. Xu, and X. Huang, "How to Fine-Tune BERT for Text Classification?," 2019, *arXiv:1905.05583*.

[8]Nargis, G. Z., & Jamil, N. (2016). Generating an emotion ontology for roman urdu text. International Journal of Computational Linguistics Research, 7(2016), 83-91.

[9]Majeed, A., Mujtaba, H., & Beg, M. O. (2020, September). Emotion detection in roman urdu text using machine learning. In Proceedings of the 35th IEEE/ACM international conference on automated software engineering (pp. 125-130).

[10]Blagus, R., & Lusa, L. (2015). Joint use of over-and under-sampling techniques and cross-validation for the development and assessment of prediction models. BMC bioinformatics, 16(1), 363.

[11]Zhu, H., Mak, D., Gioannini, J., & Xia, F. (2020). NLPStatTest: A toolkit for comparing NLP system performance. arXiv preprint arXiv:2011.13231.

Links:

[1] [\[2506.21583\] Hope Speech Detection in code-mixed Roman Urdu tweets: A Positive Turn in Natural Language Processing](#)

[2] [Roman Urdu Word Variations and Normalized Sentiment Review Dataset \(RUWV-NSR\) - Mendeley Data](#)

[3] [A dataset of Roman Urdu text with spelling variations for sentence level sentiment analysis - ScienceDirect](#)

[4] <https://www.sciencedirect.com/science/article/pii/S1877050919302200>

[5] <https://www.sciencedirect.com/science/article/pii/S1877050925010919>

- [6] https://www.researchgate.net/publication/354851655_Emotion_Detection_from_Text_Survey
- [7] <http://cips-cl.org/static/anthology/CCL-2019/CCL-19-141.pdf>
- [8] https://www.dline.info/jcl/fulltext/v7n3/jclv7n3_1.pdf
- [9] <https://sci-hub.se/10.1145/3417113.3423375>
- [10] <https://link.springer.com/article/10.1186/s12859-015-0784-9>
- [11] <https://arxiv.org/pdf/2011.13231>

13. Appendices

A. Repo Link

https://github.com/Afaf-yunas11/Roman_Urdu_Code_Mixed_Emotion_Analysis